

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas



PROYECTO

"APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE TACNA, 2026"

Curso: Construcción de Software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo	(2021070015)
Anampa Pancca David Jordan	(2022074268)

Tacna – Perú

2026

TABLA DE CONTENIDO

1. Propósito del Documento	4
2. Referencias	4
3. Requerimientos Funcionales del Sistema	5
3.1 RF-001: Autenticación de Usuario (Login Seguro)	5
3.2 RF-002: Registro de Usuarios con Consulta y Validación de DNI	10
3.3 RF-003: Recuperación y Restablecimiento de Contraseña	14
3.4 RF-004: Verificación de Cuenta mediante Correo Electrónico	16
3.5 RF-005: Registro de Reporte de Mascota Perdida (Formulario Wizard en 3 Pasos)	18
3.6 RF-006: Validación y Clasificación de Imágenes con IA Local (TensorFlow Lite)	21
3.7 RF-007: Visualización y Filtrado de Reportes de Mascotas (Feed Principal)	23
3.8 RF-008: Registro de Avistamiento de Mascota con Geolocalización GPS	25
3.9 RF-009: Algoritmo de Coincidencia Inteligente y Similitud Coseno de Embeddings	27
3.10 RF-010: Visualización de Mascotas en Mapa Interactivo (OpenStreetMap / Leaflet)	29
3.11 RF-011: Sistema de Notificaciones Push y Alertas Comunitarias (FCM)	31
3.12 RF-012: Sistema de Comentarios, Reacciones y Respuestas en Hilos	33
3.13 RF-013: Mensajería Privada y Chat en Tiempo Real entre Usuarios	35
3.14 RF-014: Generación y Exportación de Cartel de Búsqueda en Formato PDF	37
3.15 RF-015: Gestión de Perfil de Usuario	39
3.16 RF-016: Panel de Administración, Moderación de Usuarios y Control de Roles	41
3.17 RF-017: Dashboard Estadístico y Métricas Generales del Sistema	43
4. Planificación Cronológica de Tareas (Metodología SCRUM)	45
5. Casos de Uso y Escenarios del Sistema	46
6. Recursos de Software y Hardware	47
Tabla 01: Recursos de Software	47
Tabla 02: Recursos de Hardware	47
7. Evaluación de la Iteración	48
Objetivos Alcanzados	48
Objetivos No Alcanzados	48
Elementos Incluidos en la Línea Base	48
8. Conclusiones	50
9. Plan de Próximas Iteraciones	51
10. Estado del Repositorio y Trazabilidad	52

1. Propósito del Documento

El propósito de este documento es establecer de manera detallada las actividades, tareas, especificaciones de arquitectura, casos de uso, diagramas de interacción, interfaces gráficas, controles de pantalla y evidencias técnicas que se llevan a cabo durante la presente iteración correspondiente al proyecto **"Aplicación móvil colaborativa para mejorar la efectividad en la asistencia de mascotas perdidas con participación de voluntarios en la ciudad de Tacna, 2026"**.

Este plan sirve como guía formal de ingeniería para la ejecución de la iteración en el marco de la metodología ágil SCRUM y el estándar RUP/OpenUP, definiendo los objetivos específicos, criterios de aceptación medibles, modelos de datos y prototipos funcionales a implementar. Asimismo, proporciona un marco de control de calidad para verificar la trazabilidad entre los 17 requerimientos funcionales del sistema, el código fuente desarrollado en Flutter/Dart, la integración de servicios en Firebase (Authentication, Cloud Firestore, Cloud Storage, Cloud Messaging) y los modelos de visión artificial en el dispositivo móvil con TensorFlow Lite.

2. Referencias

Para la elaboración del presente Plan de Iteración se han tomado en cuenta los siguientes documentos y estándares de ingeniería:

- Documento de Especificación de Requerimientos de Software (SRS) de SOS Mascota.
- Documento de Arquitectura de Software (SAD) – Modelo de Vistas C4 y Arquitectura MVVM Reactiva.
- Plan de Desarrollo del Proyecto de Software (PDP).
- Plan de Gestión de Configuraciones y Repositorio Git.
- Plan de Aseguramiento de la Calidad del Software (SQA).
- Guía de Buenas Prácticas y Auditoría de Código Limpio de la Escuela de Ingeniería de Sistemas (UPT).

3. Requerimientos Funcionales del Sistema

A continuación se detalla la especificación exhaustiva de los **17 requerimientos funcionales** del sistema SOS Mascota, incluyendo para cada uno su ficha de caso de uso, narrativa paso a paso, flujo de excepciones, descripción de interfaz, controles de pantalla y marco de evidencias técnicas.

3.1 RF-001: Autenticación de Usuario (Login Seguro)

Descripción

Este requerimiento es esencial para la seguridad y control de acceso del sistema, ya que garantiza que únicamente usuarios autenticados (voluntarios, dueños de mascotas y administradores) puedan interactuar con la plataforma. La autenticación se basa en Firebase Authentication para el manejo criptográfico de sesiones, articulado con una verificación reactiva en tiempo real en Cloud Firestore (`auth_wrapper.dart`) para garantizar que la cuenta del usuario no se encuentre suspendida o desactivada por moderación.

Objetivos

- Autenticar credenciales de usuario mediante correo electrónico y contraseña de manera cifrada.
- Verificar en tiempo real el estado activo de la cuenta (`activo == true`) antes de autorizar la navegación.
- Diferenciar dinámicamente los roles de acceso (`usuario` vs `admin`) para dirigir al actor a su entorno correspondiente.
- Mantener la persistencia segura de la sesión tras el reinicio del aplicativo en el dispositivo móvil.

Criterios de Evaluación

- Credenciales válidas y cuenta activa otorgan acceso inmediato al feed principal o panel administrativo en menos de 2 segundos.
- Contraseña incorrecta o usuario no registrado devuelven mensajes claros y precisos sin revelar vulnerabilidades del sistema.
- Si el documento del usuario en Firestore tiene `activo == false`, el sistema cierra la sesión inmediatamente y muestra una alerta modal de bloqueo.

- La sesión se mantiene abierta entre reinicios del dispositivo sin requerir reingreso de contraseña.

Elementos de la Línea Base

- Narrativa del caso de uso de autenticación.
- Diagrama de secuencia de autenticación con verificación de estado.
- Código fuente: `lib/vista/auth/pantalla_login.dart`, `lib/vistamodelo/auth/login_vm.dart`, `lib/utls/auth_wrapper.dart`.

Caso de Uso CU-001: Iniciar Sesión

Parámetro	Especificación
Actor Principal	Usuario Voluntario / Administrador
Descripción	Permite iniciar sesión de forma segura mediante correo electrónico y contraseña validados en Firebase.
Precondición	El usuario debe estar registrado previamente en Firebase Authentication y existir en Firestore.
Poscondición	El usuario ingresa a la vista principal según su rol (<code>/inicio</code> o <code>/inicioAdmin</code>) y su token FCM se actualiza.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario abre la aplicación móvil y se ubica en la pantalla de inicio de sesión.	El sistema muestra la interfaz de login con campos para correo electrónico, contraseña, botón conmutador de visibilidad, enlace de recuperación y botón "Iniciar Sesión".
2	El usuario ingresa su correo electrónico y su contraseña, y presiona el botón "Iniciar Sesión".	El sistema activa el indicador visual de carga (<code>cargando = true</code>) e invoca a <code>LoginVM.loginYDeterminarRuta()</code> .
3		El sistema valida las precondiciones con <code>formKey.currentState!.validate()</code> .

Paso	Acción del Actor	Respuesta del Sistema
4		El sistema envía las credenciales a Firebase Authentication mediante <code>signInWithEmailAndPassword()</code> .
5		Firestore Auth valida las credenciales y devuelve la credencial con el identificador único UID.
6		El sistema consulta el documento del usuario en Firestore (<code>collection('usuarios').doc(uid)</code>).
7		El sistema comprueba que el documento exista y que el campo <code>activo</code> sea verdadero.
8		El sistema actualiza en segundo plano el token de notificaciones FCM asociado al UID.
9		El sistema redirige al usuario: si su rol es <code>admin</code> se envía a <code>/inicioAdmin</code> ; si es <code>usuario</code> se envía a <code>/inicio</code> .

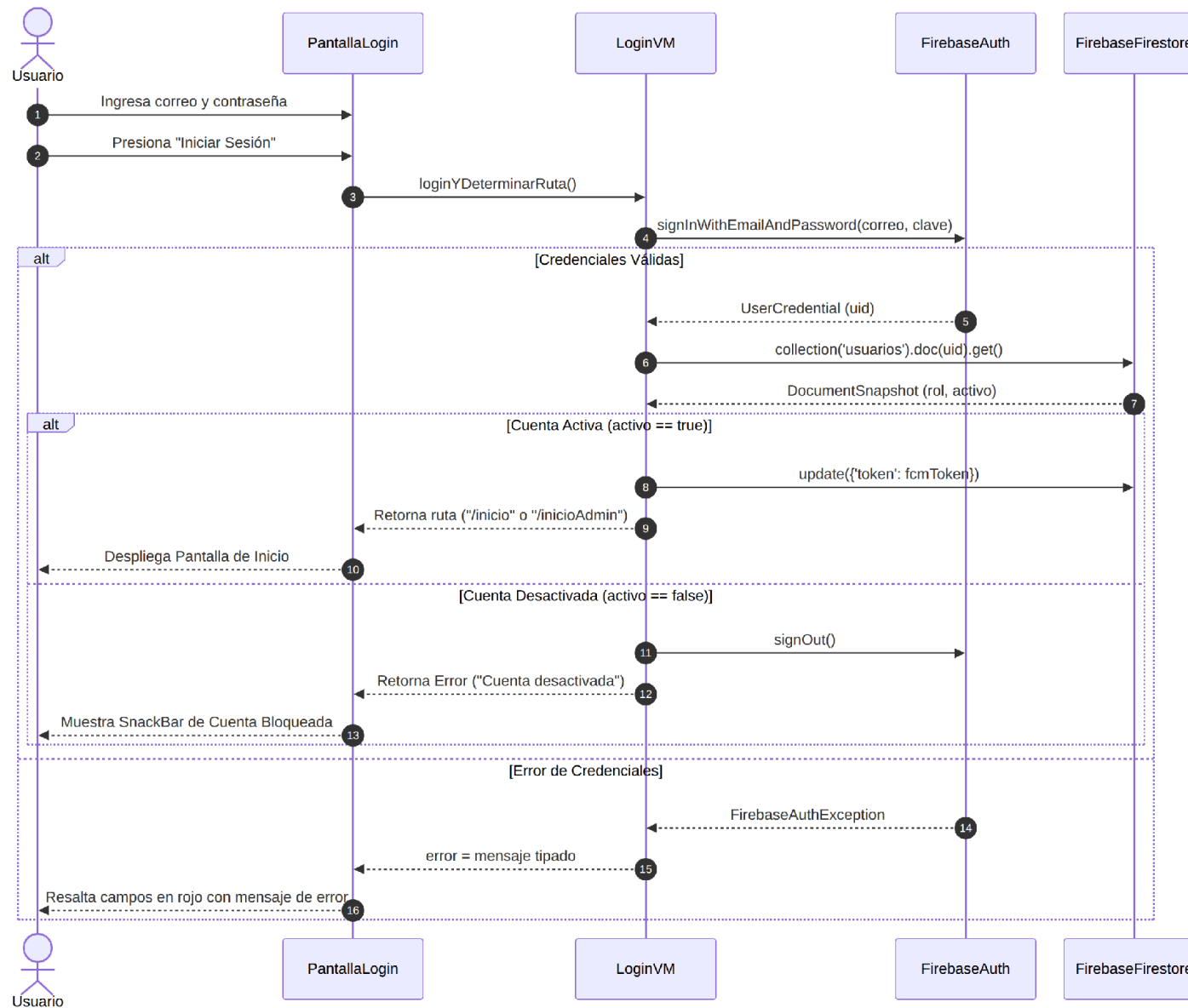
Flujo de Excepciones

Código	Condición de Fallo	Respuesta del Sistema
E1	Correo no registrado (<code>user-not-found</code>)	El sistema captura <code>FirebaseAuthException</code> , desactiva el cargando y muestra: "Usuario no existe. Verifique sus datos o regístrese".
E2	Contraseña incorrecta (<code>wrong-password</code> o <code>invalid-credential</code>)	El sistema muestra el mensaje de error: "Contraseña incorrecta". El campo de contraseña se resalta en color rojo.

Código	Condición de Fallo	Respuesta del Sistema
E3	Cuenta desactivada (activo == false)	El sistema expulsa la sesión mediante <code>FirebaseAuth.instance.signOut()</code> , redirige a la pantalla de login y despliega un SnackBar flotante rojo con el mensaje: "Tu cuenta ha sido desactivada por el administrador".
E4	Tiempo de espera agotado (sin internet)	Tras 15 segundos sin respuesta, se interrumpe la petición con <code>TimeoutException</code> y se informa: "Tiempo de espera agotado. Verifique su conexión a internet".

Fuente: Elaboración propia del equipo de trabajo.

Diagrama de Secuencia



Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/auth/pantalla_login.dart.
- **Controles Visuales:**
 - Campo de Texto txtCorreo: Validación de formato RFC 5322 con icono de sobre.
 - Campo de Texto txtClave: Campo con máscara de contraseña y botón conmutador de visibilidad.
 - Botón de Acción btnLogin: Botón elevado con indicador de progreso circular (CircularProgressIndicator) durante el procesamiento asíncrono.
 - Botón Secundario btnRecuperar: Enlace directo a PantallaRecuperar.
 - Botón Secundario btnRegistrarse: Enlace directo a PantallaRegistro.

Evidencias Técnicas Requeridas

Figura 3.1.1: Captura de Pantalla - Interfaz de Inicio de Sesión de SOS Mascota.

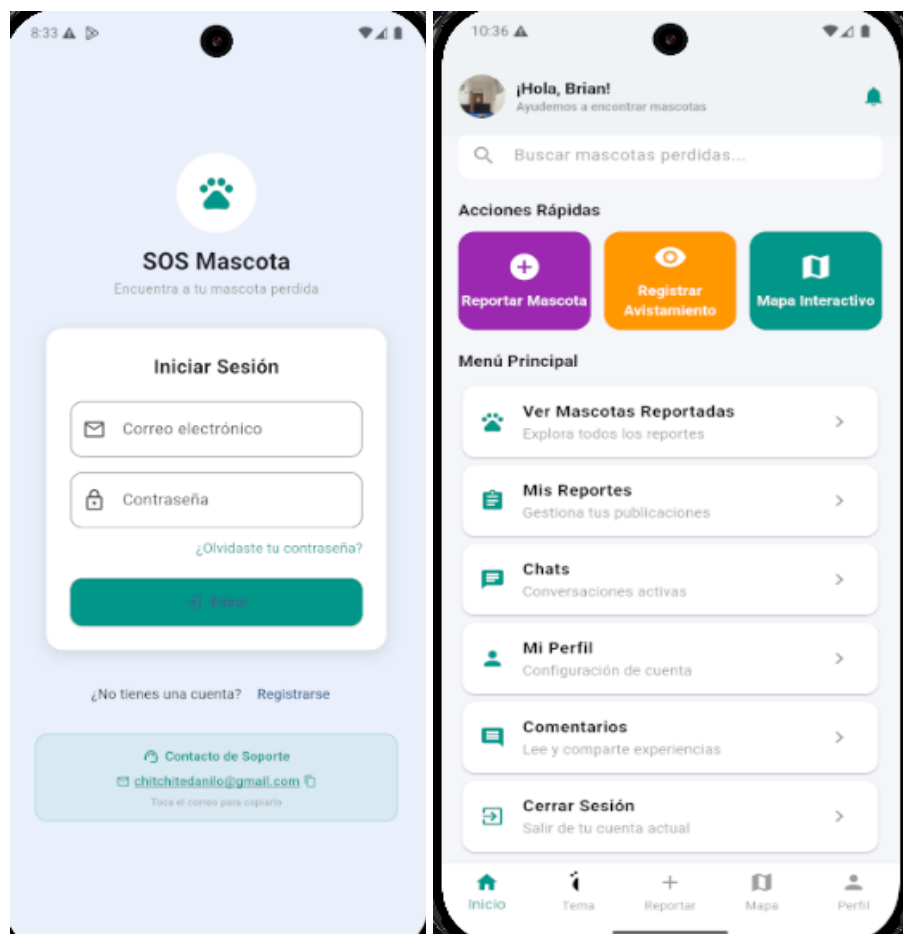


Figura 3.1.2: Captura de Pantalla - Validación de Credenciales Incorrectas y Manejo de Errores.



The screenshot displays the login interface for 'SOS Mascota'. At the top, there is a green paw print icon inside a white circle, followed by the text 'SOS Mascota' and the subtitle 'Encuentra a tu mascota perdida'. The main section is titled 'Iniciar Sesión' and contains two input fields. The first field, labeled 'Correo electrónico', contains the text 'dkdke' and is surrounded by a red border with the error message 'Correo inválido' displayed below it. The second field, labeled 'Contraseña', contains masked characters '.....' and is surrounded by a blue border. Below the password field is a link that says '¿Olvidaste tu contraseña?'. A large green button with a right arrow icon and the text 'Entrar' is positioned below the input fields. At the bottom of the login section, there is a link that says '¿No tienes una cuenta? Registrarse'. The footer section contains a 'Contacto de Soporte' link with a phone icon, an email address 'chitchitedanilo@gmail.com' with an envelope icon, and a small copy icon. Below the email address, it says 'Toca el correo para copiarlo'.

SOS Mascota
Encuentra a tu mascota perdida

Iniciar Sesión

Correo electrónico

Correo inválido

Contraseña

[¿Olvidaste tu contraseña?](#)

[→ Entrar](#)

[¿No tienes una cuenta? Registrarse](#)

[📞 Contacto de Soporte](#)
[✉ chitchitedanilo@gmail.com](mailto:chitchitedanilo@gmail.com) [📋](#)
Toca el correo para copiarlo

Figura 3.1.3: Evidencia de Base de Datos - Registro de Usuario y Token FCM en Cloud Firestore.

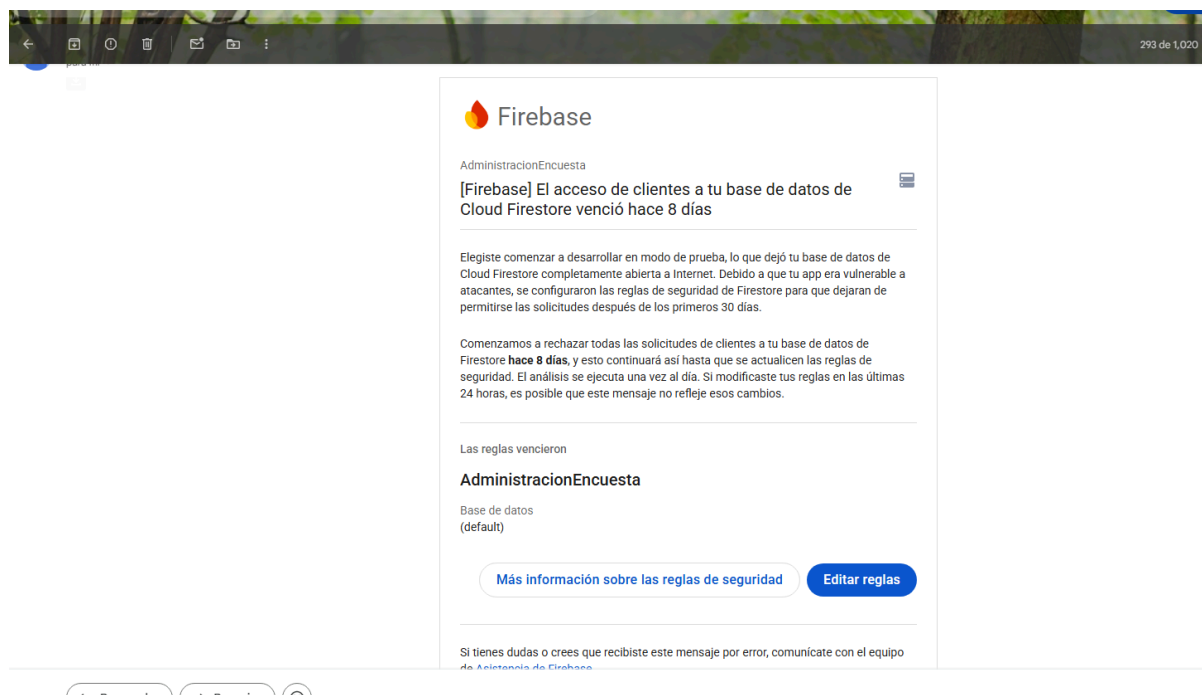
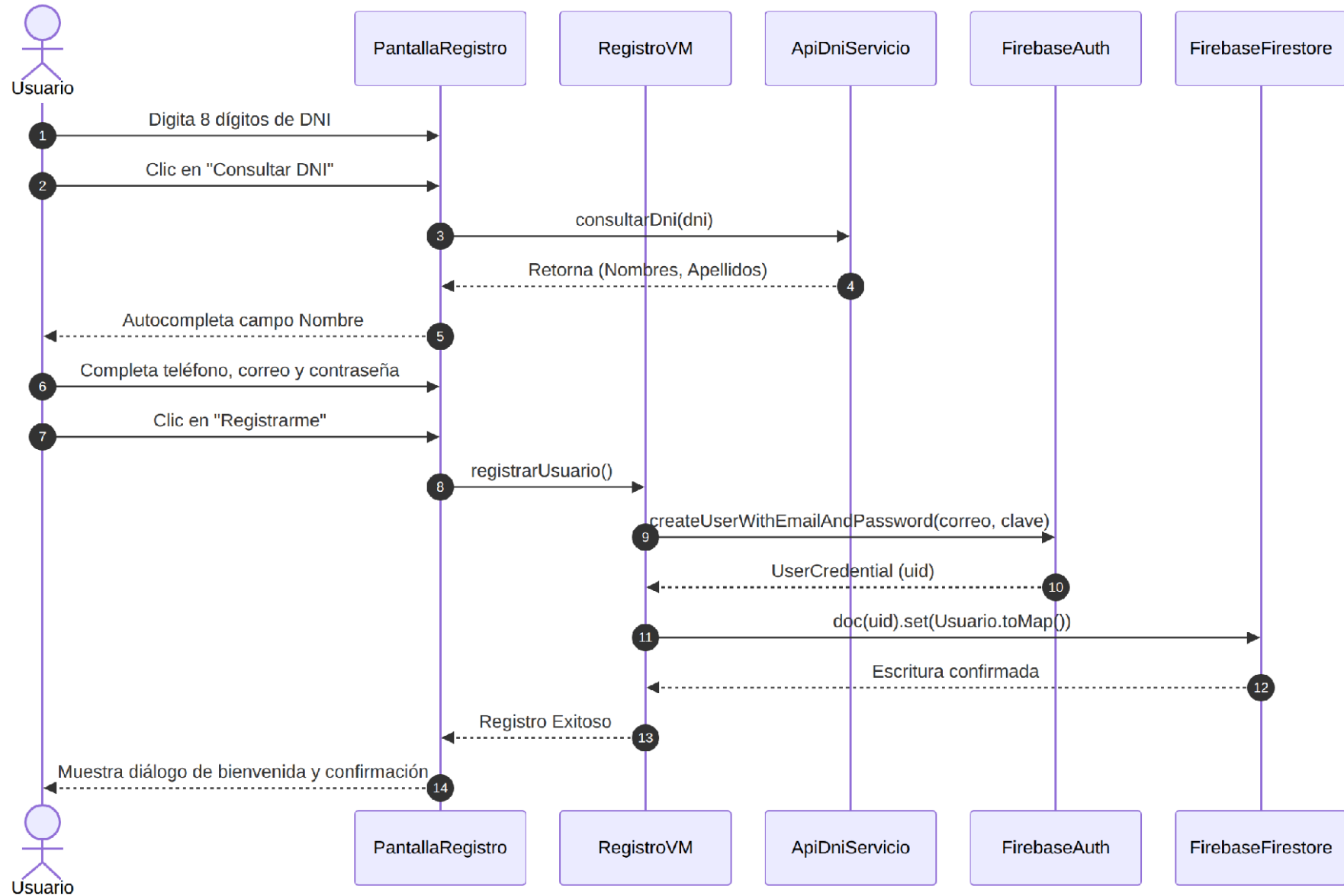


Diagrama de Secuencia



Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/auth/pantalla_registro.dart.
- **Controles Visuales:**
 - Campo Numérico txtDni: 8 dígitos, teclado numérico forzado.
 - Botón de Consulta btnConsultarDni: Desencadena consumo de API REST.
 - Campo de Solo Lectura txtNombreCompleto: Autocompletado desde API RENIEC.
 - Campo Telefónico txtTelefono: 9 dígitos comenzando con 9.
 - Campos txtCorreo, txtClave y txtConfirmarClave.
 - Botón de Envío btnRegistrar: Validador integral de formulario.

Evidencias Técnicas Requeridas

Figura 3.2.1: Captura de Pantalla - Formulario de Registro

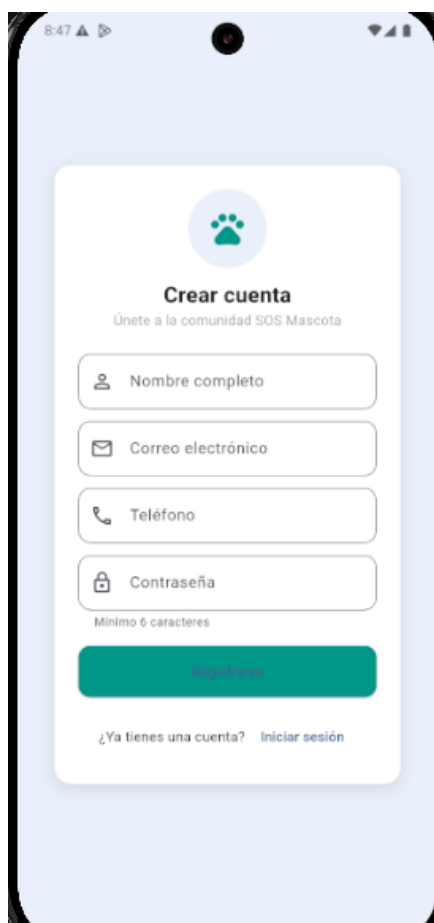


Figura 3.2.2: Evidencia de Base de Datos - Documento de Usuario Creado en Firestore con Rol usuario.

The screenshot shows the Google Cloud Firestore console interface. The breadcrumb navigation indicates the path: `sosmascota` > `Cloud Firestore` > `Base de datos (default)`. The top navigation bar includes tabs for `Datos`, `Reglas`, `Índices`, `Recuperación ante desastres`, `Uso`, and `Estadísticas de consultas`. A notification banner at the top states: "Protege tus recursos de Cloud Firestore contra los abusos, como fraudes de facturación o phishing. Configurar la Verificación de aplicaciones". Below this, there are buttons for "Más funciones en Google Cloud", "Vista del panel", and "Compilador de consultas".

The main content area displays the `usuarios` collection. On the left, a sidebar lists various collections: `avistamientos`, `chats`, `comentarios`, `notificaciones`, `notificaciones_admin`, `reportes_eliminados`, `reportes_mascotas`, and `usuarios` (which is selected). The central pane shows the `usuarios` collection with a list of document IDs. The selected document ID is `0znToGikcLT0hPyFjuTnxHJE5Fa2`. The right pane displays the document's data in a key-value format:

<code>activo</code>	<code>false</code>
<code>correo</code>	<code>"cristianconan715@gmail.com"</code>
<code>dni</code>	<code>""</code>
<code>estadoRol</code>	<code>"activo"</code>
<code>estadoVerificado</code>	<code>true</code>
<code>fcmtoken</code>	<code>null</code>
<code>fechaRegistro</code>	<code>15 de junio de 2026 a las 9:20:39 p.m. UTC-5</code>
<code>fotoPerfil</code>	<code>"https://firebasestorage.googleapis.com/v0/b/sosmascota-71d79.firebaseio.com/o/..."</code>
<code>idColaborador</code>	<code>null</code>
<code>nombre</code>	<code>"Ahmed Akhtar"</code>
<code>patacoins</code>	<code>10</code>
<code>rol</code>	<code>"usuario"</code>
<code>telefono</code>	<code>"910292214"</code>
<code>token</code>	<code>"cNAQKIQmSrGCHSZ00GVRdtAPA91bG7N-QePd05hg1s1cmjgbpwFygcDpXlw-MBPkybxJt..."</code>

At the bottom left, a status message indicates: "Ubicación de la base de datos: nam6".

3.3 RF-003: Recuperación y Restablecimiento de Contraseña

Descripción

Permite a cualquier usuario que haya olvidado su clave de acceso solicitar un enlace de restablecimiento seguro ingresando el correo electrónico asociado a su cuenta. El enlace es despachado de forma asíncrona mediante los servidores de Firebase Authentication.

Objetivos

- Proveer autoservicio de recuperación de acceso las 24 horas sin intermediación administrativa.
- Prevenir ataques de fuerza bruta limitando las solicitudes consecutivas de restablecimiento.

Criterios de Evaluación

- El sistema valida la sintaxis del correo antes de emitir la petición a Firebase.
- Correo válido recibe el enlace de recuperación en su bandeja de entrada en menos de 60 segundos.
- Se informa al usuario mediante un diálogo modal que revise su carpeta de spam o correo no deseado.

Caso de Uso CU-003: Recuperar Contraseña

Parámetro	Especificación
Actor Principal	Usuario Voluntario
Descripción	Envío de correo electrónico con enlace para restablecer la contraseña de acceso.
Precondición	La cuenta debe existir previamente en Firebase Authentication.
Poscondición	Firebase Auth despacha el correo seguro de restablecimiento.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario presiona "¿Olvidaste tu contraseña?" en PantallaLogin.	El sistema navega hacia PantallaRecuperar.

Paso	Acción del Actor	Respuesta del Sistema
2	El usuario ingresa su dirección de correo electrónico registrado.	El sistema valida el formato de correo.
3	El usuario presiona el botón "Enviar Enlace de Recuperación".	El sistema invoca <code>RecuperarVM.enviarCorreoRecuperacion()</code> .
4		El sistema invoca <code>FirebaseAuth.instance.sendPasswordResetEmail(email)</code> .
5		Firebase Auth procesa el envío del correo criptográfico.
6		El sistema muestra un diálogo de confirmación y redirige a la pantalla de login.

Flujo de Excepciones

- **E1 - Correo no registrado:** Se muestra alerta informativa: "No se encontró ninguna cuenta asociada a este correo electrónico".
- **E2 - Límite de peticiones excedido (too-many-requests):** Se bloquea temporalmente el envío y se informa: "Demasiadas solicitudes. Espere unos minutos antes de reintentar".

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/auth/pantalla_recuperar.dart`.
- **Controles Visuales:**
 - Campo de Texto `txtCorreoRecuperacion`: Validador de correo.
 - Botón `btnEnviarEnlace`: Envío asíncrono con `CircularProgressIndicator`.
 - Diálogo Modal `dlgConfirmacion`: Mensaje de éxito e instrucciones de bandeja de entrada.

Evidencias Técnicas Requeridas

Figura 3.3.1: *Captura de Pantalla - Pantalla de Recuperación de Contraseña.*

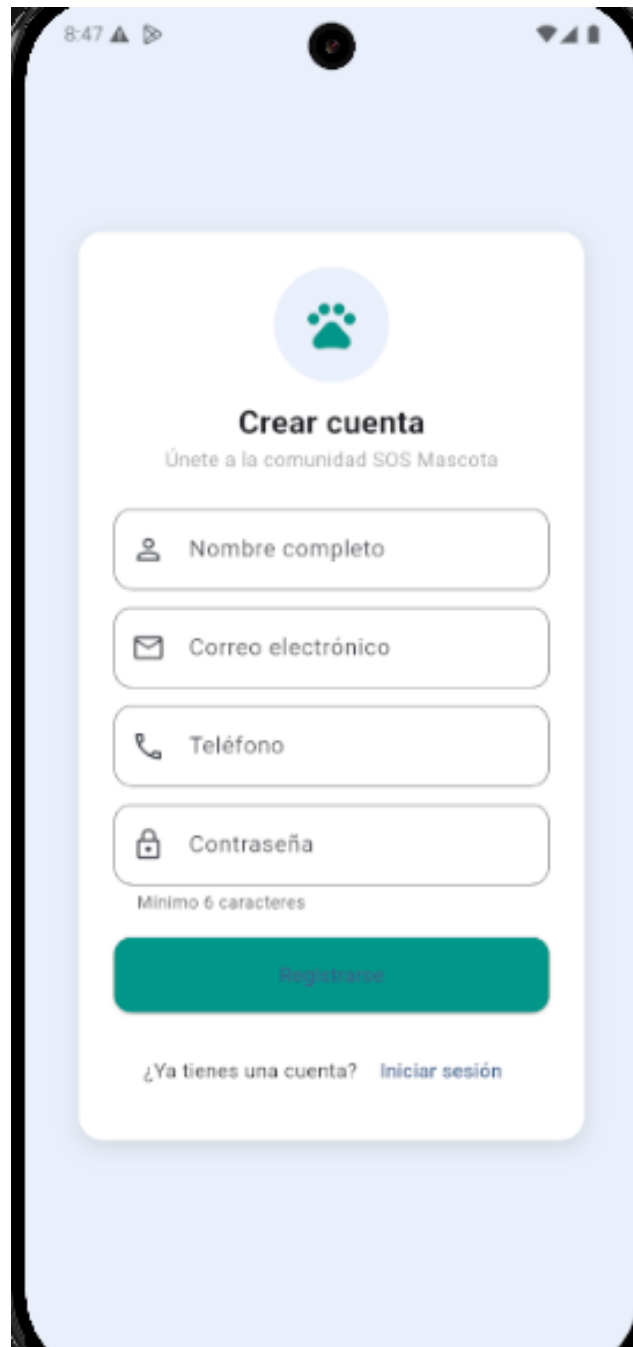


Figura 3.3.2: Captura de Pantalla - Diálogo de Envío Exitoso de Correo de Restablecimiento.





Recuperar contraseña

Ingresa tu correo y te enviaremos un enlace para restablecer tu contraseña

Correo electrónico

 ristianconan715@gmail.com

Enviar enlace de recuperación

¿Recordaste tu contraseña? [Iniciar sesión](#)

3.4 RF-004: Verificación de Cuenta mediante Correo Electrónico

Descripción

Envía un mensaje de confirmación con enlace criptográfico al correo del usuario recién registrado, impidiendo que cuentas sin validar puedan publicar recompensas económicas o adoptar animales en la comunidad.

Objetivos

- Eliminar el registro masivo de cuentas con correos falsos o inexistentes.
- Validar la propiedad de la casilla de correo electrónico del voluntario.

Criterios de Evaluación

- El sistema detecta en tiempo real cuando el enlace ha sido pulsado mediante un sondeo periódico de `currentUser.reload()`.
- Redirige automáticamente al feed principal una vez verificada la cuenta.

Caso de Uso CU-004: Verificar Correo Electrónico

Parámetro	Especificación
Actor Principal	Nuevo Usuario Registrado
Descripción	Verificación obligatoria de propiedad de correo electrónico.
Precondición	Usuario recién creado con <code>emailVerified == false</code> .
Poscondición	La cuenta cambia a estado verificado y accede al sistema.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario culmina el registro de cuenta.	El sistema detecta <code>emailVerified == false</code> y abre <code>PantallaVerificaEmail</code> .
2		El sistema despacha el correo de verificación e inicia un temporizador de sondeo cada 3 segundos.

Paso	Acción del Actor	Respuesta del Sistema
3	El usuario abre su aplicación de correo y presiona el enlace de verificación recibido.	Firebase Authentication actualiza el atributo <code>emailVerified = true</code> .
4		El temporizador de la app detecta la actualización al ejecutar <code>currentUser.reload()</code> .
5		El sistema cancela el temporizador y navega automáticamente al menú principal.

Flujo de Excepciones

- **E1 - Correo no recibido:** El usuario puede pulsar el botón "Reenviar correo" transcurrido el tiempo de espera de 60 segundos.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/auth/pantalla_verifica_email.dart`.
- **Controles Visuales:**
 - Texto descriptivo con la dirección de correo a la que fue enviado el enlace.
 - Botón `btnReenviarEmail`: Con temporizador de enfriamiento de 60 segundos.
 - Botón `btnCancelar`: Cierra sesión y retorna al login.

Evidencias Técnicas Requeridas

Figura 3.4.1: Captura de Pantalla - Pantalla de Espera de Verificación de Correo Electrónico.

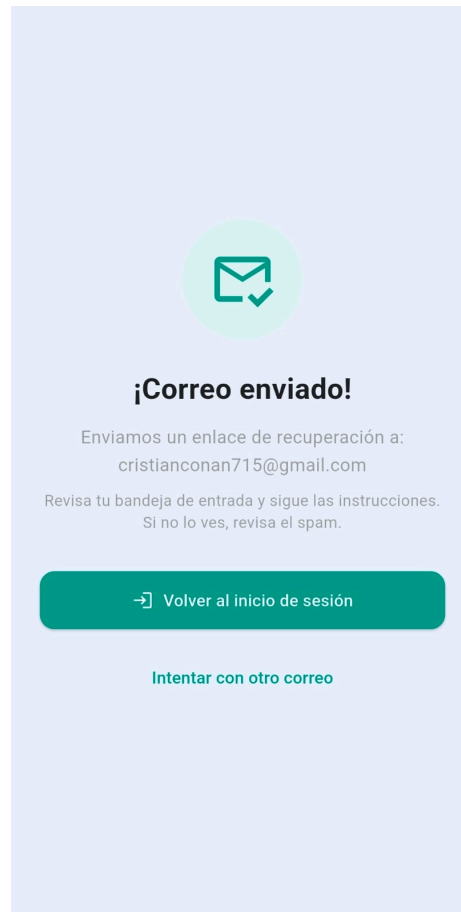
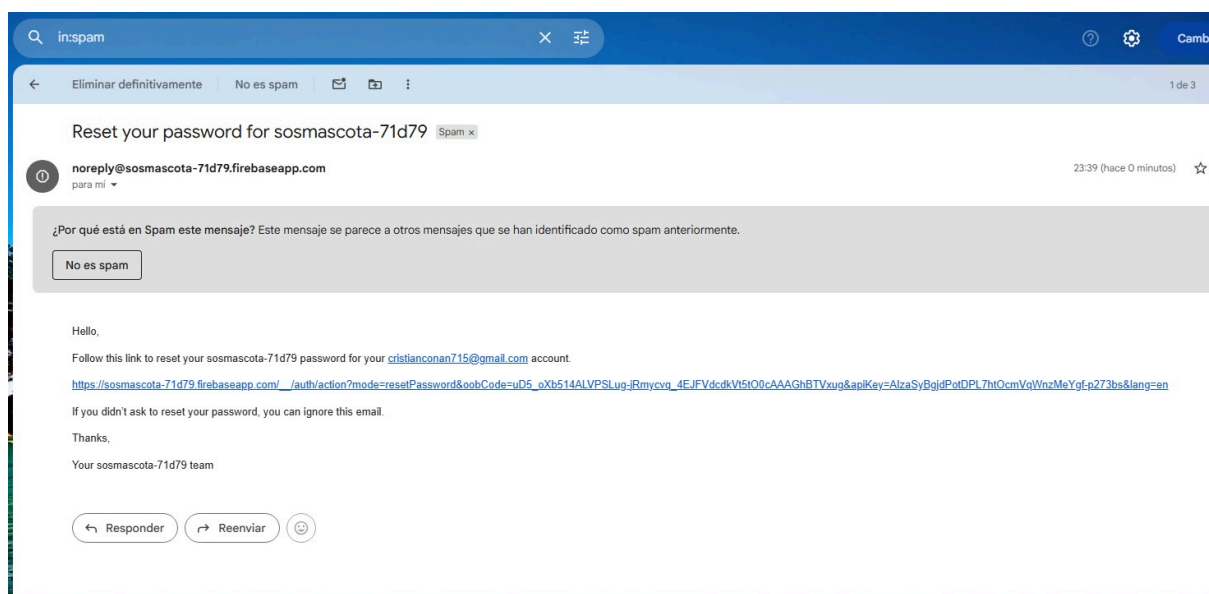


Figura 3.4.2: Captura de Pantalla - Mensaje de Verificación Emitido por Firebase Auth en Bandeja de Correo.



3.5 RF-005: Registro de Reporte de Mascota Perdida (Formulario Wizard en 3 Pasos)

Descripción

Permite a los usuarios dueños de mascotas registrar formalmente la desaparición de un animal mediante un formulario interactivo estructurado en tres etapas consecutivas (Wizard): Datos del animal y fotografía, Localización y circunstancias, y Recompensa con confirmación.

Objetivos

- Reducir la carga cognitiva del usuario organizando el proceso en pasos lógicos secuenciales.
- Almacenar fotografías de alta resolución en Firebase Storage vinculadas al identificador del reporte.
- Capturar las coordenadas exactas de latitud y longitud donde el animal fue visto por última vez en Tacna.

Caso de Uso CU-005: Registrar Mascota Perdida

Parámetro	Especificación
Actor Principal	Dueño de Mascota / Usuario Autenticado
Descripción	Creación y publicación de una alerta de búsqueda geolocalizada en Tacna.
Precondición	Usuario con sesión activa y permisos de cámara/galería concedidos.
Poscondición	El reporte se almacena en Firestore, la imagen en Storage y se despacha notificación push comunitaria.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario presiona el botón flotante "Reportar Mascota Perdida" en el menú principal.	El sistema abre PantallaReporteMascota posicionándose en el Paso 1: Datos de la Mascota.

Paso	Acción del Actor	Respuesta del Sistema
2	El usuario selecciona una foto desde su galería o toma una foto con la cámara.	El sistema invoca <code>ImagenUtil.validarYSubirFoto()</code> preprocesando la imagen con IA local (TFLite).
3	El usuario ingresa nombre, especie (perro, gato, otro), raza, sexo y señas particulares.	El sistema valida que los campos descriptivos obligatorios no estén vacíos.
4	El usuario presiona "Continuar".	El sistema valida el formulario del Paso 1 y avanza al Paso 2: Ubicación y Fecha .
5	El usuario selecciona el distrito de Tacna (ej. Gregorio Albarracín), dirección y toca el mapa para fijar el marcador GPS.	El sistema captura <code>latitud</code> y <code>longitud</code> , y formatea la fecha y hora de la pérdida.
6	El usuario presiona "Continuar".	El sistema avanza al Paso 3: Recompensa y Confirmación .
7	El usuario define la recompensa en dinero (S/.)	El sistema muestra el resumen general con el afiche preliminar de la mascota.
8	El usuario presiona "Publicar Reporte".	El sistema invoca <code>ReporteMascotaVM.guardarReporte()</code> , guarda en Firestore, dispara una notificación push global y redirige al feed.

Flujo de Excepciones

- **E1 - Fotografía no válida:** Si TFLite determina que no hay un animal con confianza ≥ 0.60 , se bloquea el avance y se muestra: "La imagen no corresponde a una mascota válida".
- **E2 - Coordenadas no seleccionadas:** Se impide avanzar al paso 3 hasta que el usuario fije un punto en el mapa de Tacna.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/reportes/pantalla_reporte_mascota.dart`.

- **Controles Visuales:**

- Indicador de Progreso Wizard: Pasos 1, 2 y 3 con numeración activa.
- Selector de Imagen: Botón con previsualización circular de la fotografía.
- Controles de Entrada: Selectores desplegables (Dropdown) para especie, sexo y distrito.
- Widget de Mapa: Vista interactiva para fijar marcador geográfico de pérdida.
- Botones de Navegación: "Anterior", "Siguiente" y "Publicar Reporte".

Evidencias Técnicas Requeridas

Figura 3.5.1: *Captura de Pantalla - Wizard Paso 1: Datos de la Mascota y Carga de Fotografía.*

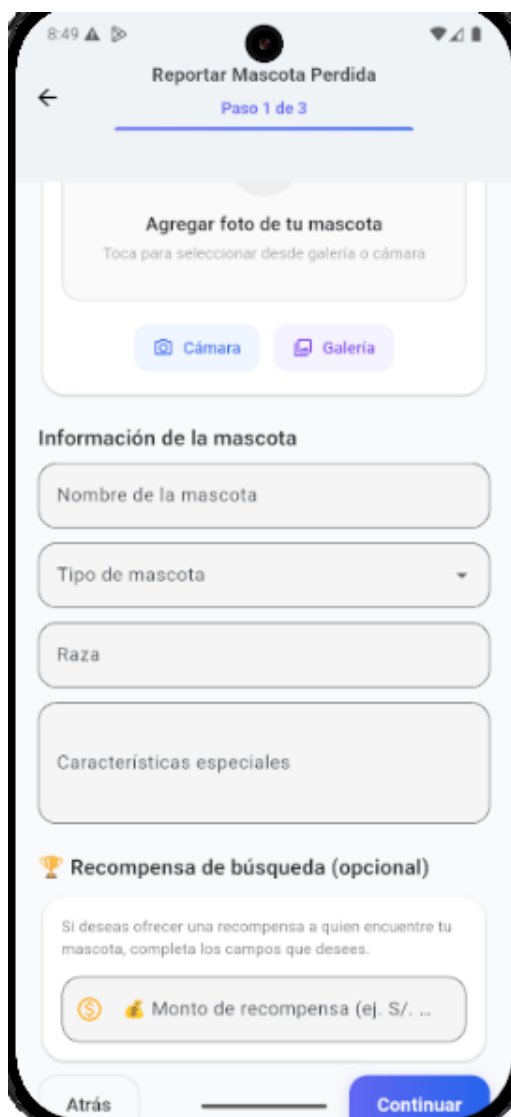


Figura 3.5.2: *Captura de Pantalla - Wizard Paso 2: Selección Geográfica en el Mapa de Tacna.*

The screenshot shows a mobile application interface for reporting a lost pet. The title is 'Reportar Mascota Perdida' and it is 'Paso 2 de 3'. The section is 'Lugar y momento de pérdida'. It includes input fields for 'Fecha de pérdida' (with a calendar icon) and 'Hora aproximada' (with a clock icon). Below these is a section 'Seleccionar ubicación en el mapa' with a button 'Abrir mapa interactivo'. Further down are input fields for 'Dirección (puedes editar si lo deseas)' (with a location pin icon), 'Puntos de referencia', and '¿Cómo se perdió?'. At the bottom are 'Atrás' and 'Continuar' buttons.

8:50

Reportar Mascota Perdida

Paso 2 de 3

Lugar y momento de pérdida

Fecha de pérdida

Hora aproximada

Seleccionar ubicación en el mapa

Abrir mapa interactivo

Dirección (puedes editar si lo deseas)

Puntos de referencia

¿Cómo se perdió?

Atrás

Continuar

Figura 3.5.3: *Captura de Pantalla - Wizard Paso 3: Asignación de Recompensa y Confirmación.*

The screenshot displays the 'Reportar Mascota Perdida' (Report Lost Pet) app interface at the final step of a three-part wizard. The title bar at the top shows the time as 8:50 and the status 'Paso 3 de 3'. Below the title, a section titled 'Resumen del reporte' (Report Summary) features a placeholder image of a golden retriever puppy. The summary includes the following details: Name: asdasd; Type: Perro • perro; Lost: 01/07/2026 a las 16:50; Address: 902, Calle La Mar, Urbanización Los Cipreses, Leoncio Prado, Jesús María, Tacna, 23001, Perú; Reference: asdasda; and Special characteristics: asdad. A reward section, 'Recompensa de búsqueda' (Search reward), is set to a 'Monto: S/. 100'. At the bottom, there are two buttons: 'Atrás' (Back) and 'Publicar reporte' (Publish report).

8:50

Reportar Mascota Perdida

Paso 3 de 3

Resumen del reporte



Nombre: asdasd

Tipo: Perro • perro

Perdido: 01/07/2026 a las 16:50

Dirección: 902, Calle La Mar, Urbanización Los Cipreses, Leoncio Prado, Jesús María, Tacna, 23001, Perú

Referencia: asdasda

Características especiales: asdad

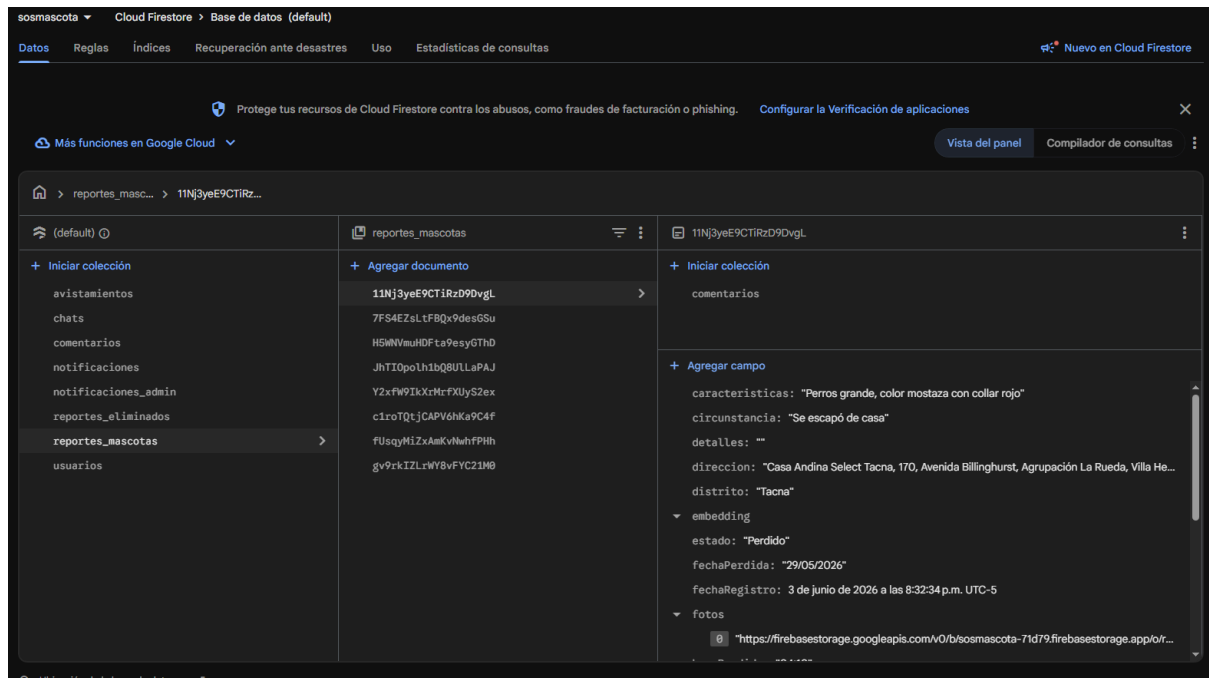
 **Recompensa de búsqueda**

 **Monto: S/. 100**

Atrás

Publicar reporte

Figura 3.5.4: Evidencia de Base de Datos - Registro de Mascota en la Colección *reportes* de Firestore.



3.6 RF-006: Validación y Clasificación de Imágenes con IA Local (TensorFlow Lite)

Descripción

Ejecuta de manera autónoma un modelo de visión computacional convolucional (MobileNet) en el propio teléfono móvil sin requerir internet para la inferencia. Analiza cada fotografía seleccionada por el usuario, detecta la presencia de mascotas (perros/gatos) y calcula el porcentaje de confianza, rechazando imágenes que no correspondan a animales.

Objetivos

- Blindar la base de datos contra la subida de imágenes basura, memes o fotos humanas.
- Extraer el vector matemático de 1280 dimensiones (embeddings) para el cotejo biométrico automatizado.

Criterios de Evaluación

- Inferencia ejecutada en menos de 500 ms en dispositivos de gama media.
- Confianza mínima requerida del 60% (0.60). Si es menor, se lanza `ValidationException`.

Caso de Uso CU-006: Inferencia de Visión Computacional On-Device

Parámetro	Especificación
Actor Principal	Motor de Inteligencia Artificial (TFLite)
Descripción	Clasificación de imagen y extracción de vector de características numéricas en el dispositivo.
Precondición	Imagen seleccionada y modelo .tflite cargado en memoria RAM.
Poscondición	Retorna boolean de validación y vector de embeddings de 1280 dimensiones.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario selecciona una fotografía desde la galería o cámara.	El sistema invoca <code>ImagenUtil.validarYSubirFoto()</code> .

Paso	Acción del Actor	Respuesta del Sistema
2		ServicioTFLite redimensiona la imagen a 224x224 píxeles y normaliza valores RGB a matriz flotante [1, 224, 224, 3].
3		El intérprete ejecuta la inferencia de clasificación canina/felina.
4		Si la confianza supera el umbral de 0.60, se ejecuta el modelo extractor de embeddings.
5		Se genera el vector numérico de 1280 flotantes y se adjunta a la carga útil.
6		La imagen se sube a Firebase Storage y el proceso continúa exitosamente.

Flujo de Excepciones

- **E1 - Objeto no identificado o humano:** El clasificador arroja confianza menor a 0.60. Se interrumpe la subida con mensaje modal de advertencia al usuario.

Evidencias Técnicas Requeridas

Figura 3.6.1: Captura de Pantalla - Alerta de Imagen Rechazada por Clasificador de Inteligencia Artificial.

The screenshot displays the 'Reportar Mascota Perdida' (Report Lost Pet) app interface. At the top, there is a back arrow and the title 'Reportar Mascota Perdida', followed by a progress indicator 'Paso 1 de 3'. The main section is titled 'Foto de tu mascota' (Photo of your pet). It features a large rectangular area with a camera icon and the text 'Agregar foto de tu mascota' (Add photo of your pet), with a subtext 'Toca para seleccionar desde galería o cámara' (Tap to select from gallery or camera). Below this are two buttons: 'Cámara' (Camera) and 'Galería' (Gallery). Underneath the photo section is the 'Información de la mascota' (Pet information) section, which includes input fields for 'Nombre de la mascota' (Pet name) and 'Tipo de mascota' (Pet type). At the bottom, a red banner displays an error message: '❌ La imagen no parece contener un perro o gato válido. Por favor, sube una imagen clara de la mascota perdida.' (The image does not seem to contain a valid dog or cat. Please upload a clear image of the lost pet.)

3.7 RF-007: Visualización y Filtrado de Reportes de Mascotas (Feed Principal)

Descripción

Despliega en tiempo real la lista cronológica de todas las mascotas extraviadas en la región de Tacna mediante una cuadrícula de tarjetas interactivas. Cada tarjeta incluye la imagen principal, el nombre del animal, el distrito, el tiempo transcurrido (hace X horas), el estado ("Perdido" o "Encontrado") y botones de acceso directo a la ficha completa, mapa y comentarios.

Objetivos

- Facilitar a la comunidad la identificación rápida de animales en búsqueda activa.
- Permitir filtros reactivos por distrito, especie (perros/gatos) y estado de resolución.

Criterios de Evaluación

- Los nuevos reportes publicados por otros usuarios aparecen en la pantalla sin necesidad de recargar la aplicación (Stream reactivo).
- El filtrado por distrito se realiza en memoria en menos de 100 ms.

Caso de Uso CU-007: Explorar y Filtrar Reportes

Parámetro	Especificación
Actor Principal	Voluntario / Usuario General
Descripción	Visualización reactiva y filtrado multicriterio de las alertas de mascotas extraviadas.
Precondición	Conexión a internet y servicio Firestore accesible.
Poscondición	La interfaz despliega la lista filtrada de mascotas activas.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario ingresa a la pestaña "Reportes" en el menú inferior.	El sistema abre <code>PantallaVerReportes</code> y suscribe un <code>Stream</code> a la colección <code>reportes</code> .

Paso	Acción del Actor	Respuesta del Sistema
2		El sistema renderiza las tarjetas de mascotas con imagen, nombre, distrito y tiempo transcurrido.
3	El usuario pulsa el chip de filtro "Gatos" y selecciona el distrito "Tacna Centro".	El sistema filtra la lista en memoria y actualiza la cuadrícula al instante.
4	El usuario toca la tarjeta de una mascota específica.	El sistema navega a la vista de detalle con opciones de cartel PDF, comentarios y mapa.

Flujo de Excepciones

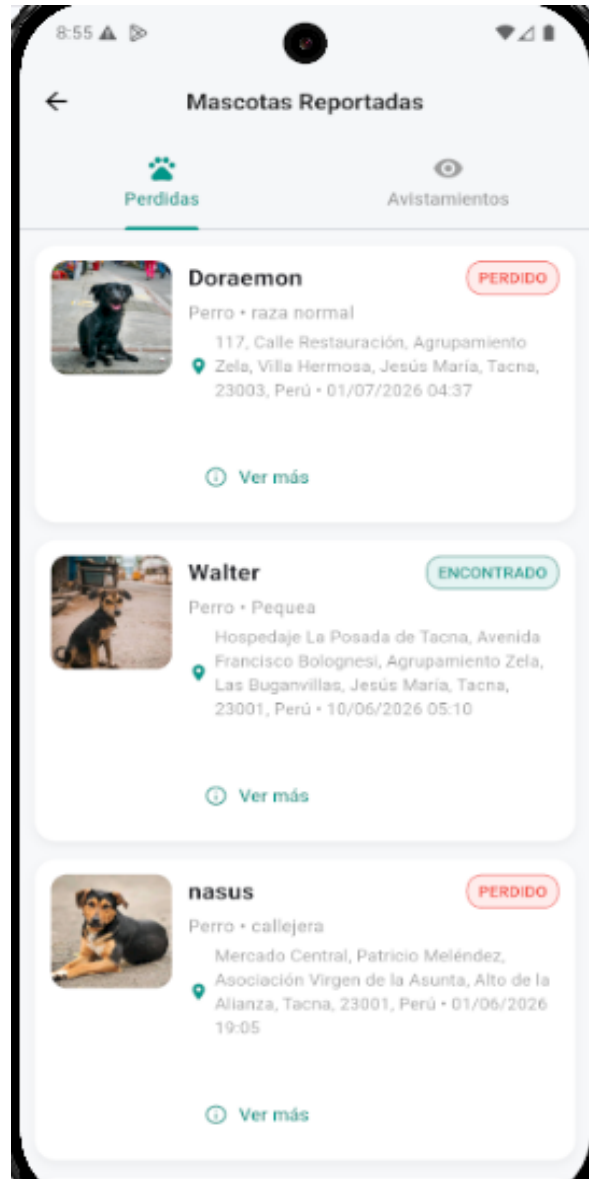
- **E1 - No hay reportes que coincidan:** El sistema muestra una vista vacía amigable con el mensaje: "No se encontraron mascotas extraviadas con los filtros seleccionados".

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/reportes/pantalla_ver_reportes.dart`.
- **Controles Visuales:**
 - Barra de Búsqueda de Texto: Búsqueda por nombre o raza.
 - Chips de Filtrado Rápido: "Todos", "Perros", "Gatos", "Otros".
 - Selector Desplegable de Distritos: Lista de los distritos de la provincia de Tacna.
 - Tarjetas de Mascota: Componente con imagen en caché (`CachedNetworkImage`), insignia de estado y botón de compartir.

Evidencias Técnicas Requeridas

Figura 3.7.1: Captura de Pantalla - Feed Principal de Reportes de Mascotas Reportadas.



3.8 RF-008: Registro de Avistamiento de Mascota con Geolocalización GPS

Descripción

Permite a cualquier voluntario que observe una mascota desorientada en las calles de Tacna capturar una fotografía instantánea, obtener la posición GPS exacta del dispositivo móvil y registrar el avistamiento sin necesidad de conocer la identidad del dueño.

Objetivos

- Capturar avistamientos en menos de 15 segundos en situaciones de movilidad en la vía pública.

Criterios de Evaluación

- Obtención de coordenadas con precisión menor a 15 metros mediante GPS satelital.

Caso de Uso CU-008: Registrar Avistamiento en Vía Pública

Parámetro	Especificación
Actor Principal	Voluntario en Vía Pública
Descripción	Registro fotográfico y geolocalizado de un animal presuntamente extraviado.
Precondición	Sensor GPS activado y permisos de cámara concedidos.
Poscondición	Se guarda el avistamiento en Firestore

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El voluntario observa un animal en la vía pública y pulsa "Registrar Avistamiento".	El sistema abre PantallaAvistamiento y activa la geolocalización en segundo plano.
2	El voluntario presiona el botón de cámara y toma la fotografía del animal.	El sistema valida la fotografía con IA local (TFLite) y extrae sus coordenadas GPS.

Paso	Acción del Actor	Respuesta del Sistema
3	El voluntario escribe una descripción breve (ej. "Cerca a la plaza de Pocollay, collar rojo").	El sistema habilita el botón "Guardar Avistamiento".
4	El voluntario presiona "Guardar Avistamiento".	AvistamientoVM.guardarAvistamiento() persiste los datos en Firestore y Storage.
5		El sistema abona automáticamente.
6		El sistema dispara en segundo plano el algoritmo de coincidencia inteligente (RF-009).
7		Se muestra mensaje modal de agradecimiento y retorno al mapa interactivo.

Flujo de Excepciones

- **E1 - GPS apagado:** El sistema despliega un diálogo solicitando encender el servicio de ubicación del dispositivo antes de continuar.
- **E2 - Descripción vacía:** Se resalta el campo de descripción con la advertencia de campo obligatorio.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/reportes/pantalla_avistamiento.dart.
- **Controles Visuales:**
 - Cuadro de Previsualización de Foto: Con botón flotante de obturador.
 - Tarjeta de Coordenadas GPS: Muestra latitud, longitud y precisión en metros.
 - Campo de Texto txtDescripcion: Entrada multilínea de detalles del avistamiento.
 - Botón de Guardado: Con animación de confirmación.

Evidencias Técnicas Requeridas

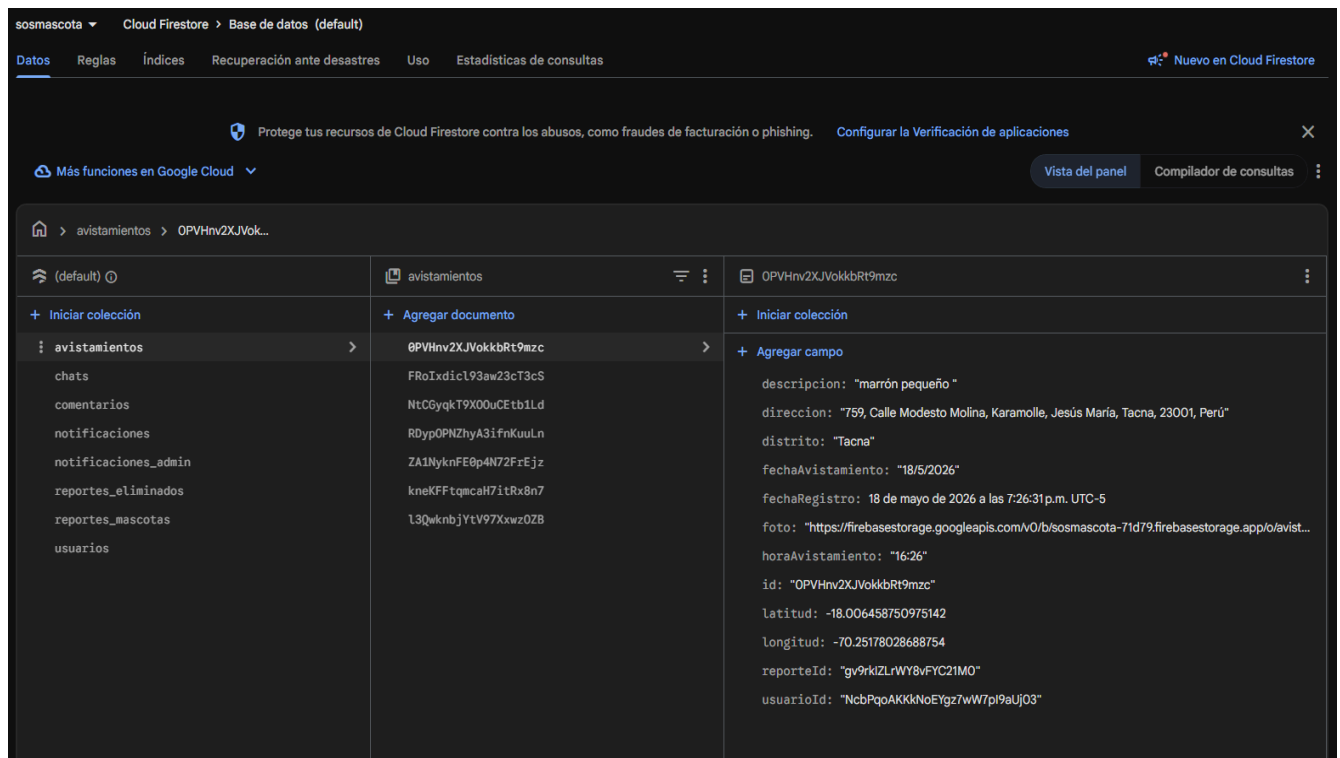
Figura 3.8.1: *Captura de Pantalla - Formulario de Avistamiento Rápido con Captura de Coordenadas GPS.*

The screenshot shows a mobile application interface for registering a sighting. The title bar at the top says "Registrar Avistamiento". The form consists of several sections:

- Agregar foto del avistamiento:** A section with a camera icon and the text "Toca para seleccionar desde galería o cámara". Below it are two buttons: "Cámara" (with a camera icon) and "Galería" (with a gallery icon).
- Ubicación del avistamiento:** A section with a location pin icon and a button labeled "Seleccionar desde el mapa" with a map icon.
- Fecha y hora del avistamiento:** A section with a clock icon and two buttons: "Fecha" (with a calendar icon) and "Hora" (with a clock icon).
- Descripción del avistamiento:** A section with a notepad icon and a text input field. The placeholder text is "Ejemplo: perro marrón pequeño, se enc...".

At the bottom of the form is a large orange button labeled "Guardar Avistamiento".

Figura 3.8.2: *Evidencia de Base de Datos - Documento Creado en la Colección avistamientos.*



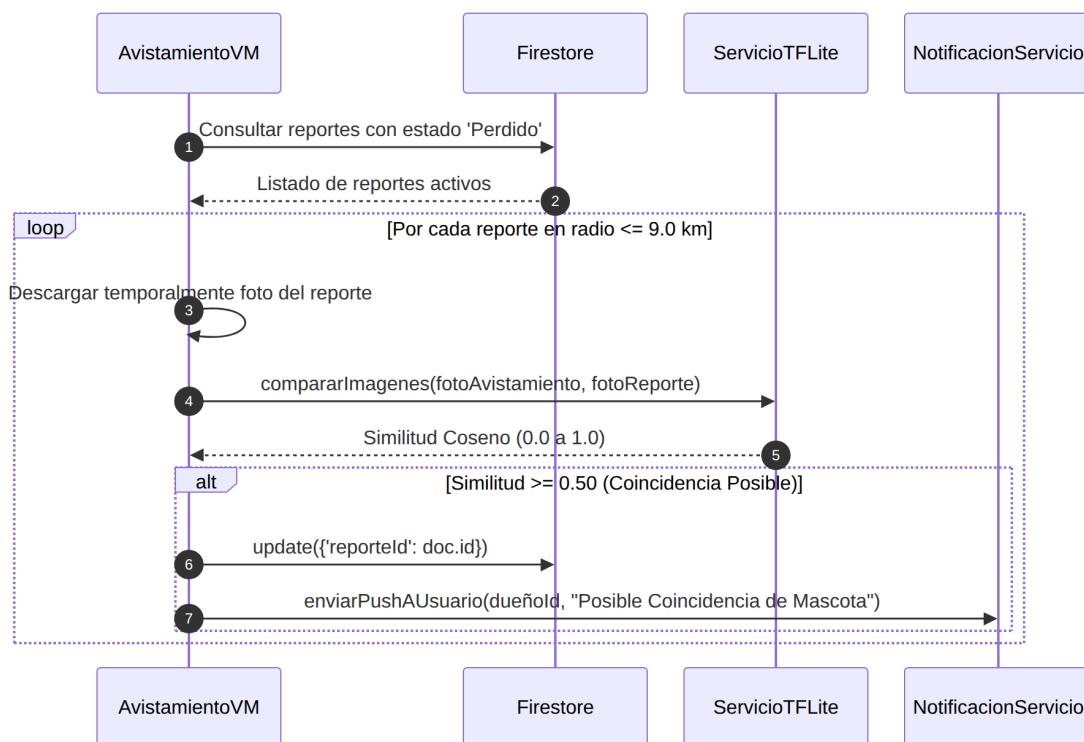
3.9 RF-009: Algoritmo de Coincidencia Inteligente y Similitud Coseno de Embeddings

Descripción

Algoritmo de cruce biométrico y geográfico que se dispara automáticamente cada vez que se guarda un nuevo avistamiento:

1. Filtra los reportes activos en un radio geográfico euclidiano/Haversine menor o igual a **9.0 kilómetros**.
2. Compara el vector de características del avistamiento con los reportes en el radio mediante **Similitud Coseno**.
3. Si la similitud supera el umbral del **50% (0.50)**, asocia el avistamiento al reporte y despacha una notificación push inmediata al dueño de la mascota indicándole que su animal pudo haber sido encontrado.

Diagrama de Secuencia del Algoritmo



Criterios de Evaluación

- Tiempo de ejecución del cotejo inferior a 3 segundos para hasta 50 reportes activos en el radio.
- Vinculación automática en Firestore mediante el campo `reporteId`.

- Notificación push despachada exclusivamente al dueño del animal coincidentemente identificado.

Evidencias Técnicas Requeridas

Figura 3.9.1: *Captura de Pantalla - Notificación Push de Posible Coincidencia Recibida por el Dueño.*

Reportar Mascota Perdida

←

Paso 1 de 3

Foto de tu mascota



Cámara

Galería

Información de la mascota

Nombre de la mascota

Tipo de mascota

Raza

✓ Imagen válida detectada y subida correctamente.

3.10 RF-010: Visualización de Mascotas en Mapa Interactivo (OpenStreetMap / Leaflet)

Descripción

Presenta un mapa georreferenciado de la provincia de Tacna con marcadores dinámicos diferenciados por color:

- **Marcadores Azules:** Ubicaciones donde los dueños perdieron a sus mascotas.
- **Marcadores Naranjas:** Puntos exactos donde voluntarios registraron avistamientos recientes.

Al pulsar un marcador, se despliega una tarjeta flotante con la foto en miniatura, nombre, fecha y botón para trazar la ruta de navegación.

Objetivos

- Brindar una perspectiva espacial interactiva de las zonas calientes de extravío en Tacna.
- Facilitar rutas de patrullaje a brigadas voluntarias de rescate.

Caso de Uso CU-010: Navegar en el Mapa Interactivo

Parámetro	Especificación
Actor Principal	Voluntario / Ciudadano
Descripción	Exploración cartográfica de reportes y avistamientos en Tacna.
Precondición	Conexión a internet para descarga de mosaicos cartográficos.
Poscondición	La vista despliega pines georreferenciados con fichas emergentes.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario selecciona la opción "Mapa" en la barra de navegación.	El sistema abre PantallaMapaInteractivo centrando la vista en Tacna (-18.0146, -70.2536).
2		El sistema descarga los marcadores de reportes y avistamientos desde Firestore.

Paso	Acción del Actor	Respuesta del Sistema
3	El usuario pulsa un marcador de color naranja (avistamiento).	El sistema abre un panel inferior (BottomSheet) con la foto, fecha y notas del voluntario.
4	El usuario presiona el botón "Cómo llegar".	El sistema abre la aplicación nativa de mapas (Google Maps) con las coordenadas de destino.

Flujo de Excepciones

- **E1 - Sin conexión:** Se muestran los marcadores utilizando la caché local de mosaicos sin interrumpir la navegación.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/mapa/pantalla_mapa_interactivo.dart.
- **Controles Visuales:**
 - Vista de Mapa FlutterMap: Motor de renderizado cartográfico con soporte gestual (zoom, rotación, paneo).
 - Marcadores Personalizados: Pines vectoriales azules (reportes) y naranjas (avistamientos).
 - Modal Emergente BottomSheet: Resumen del reporte con miniatura y acceso a ficha técnica.

Evidencias Técnicas Requeridas

Figura 3.10.1: *Captura de Pantalla - Mapa Cartográfico de Tacna con Marcadores de Pérdida y Avistamiento.*

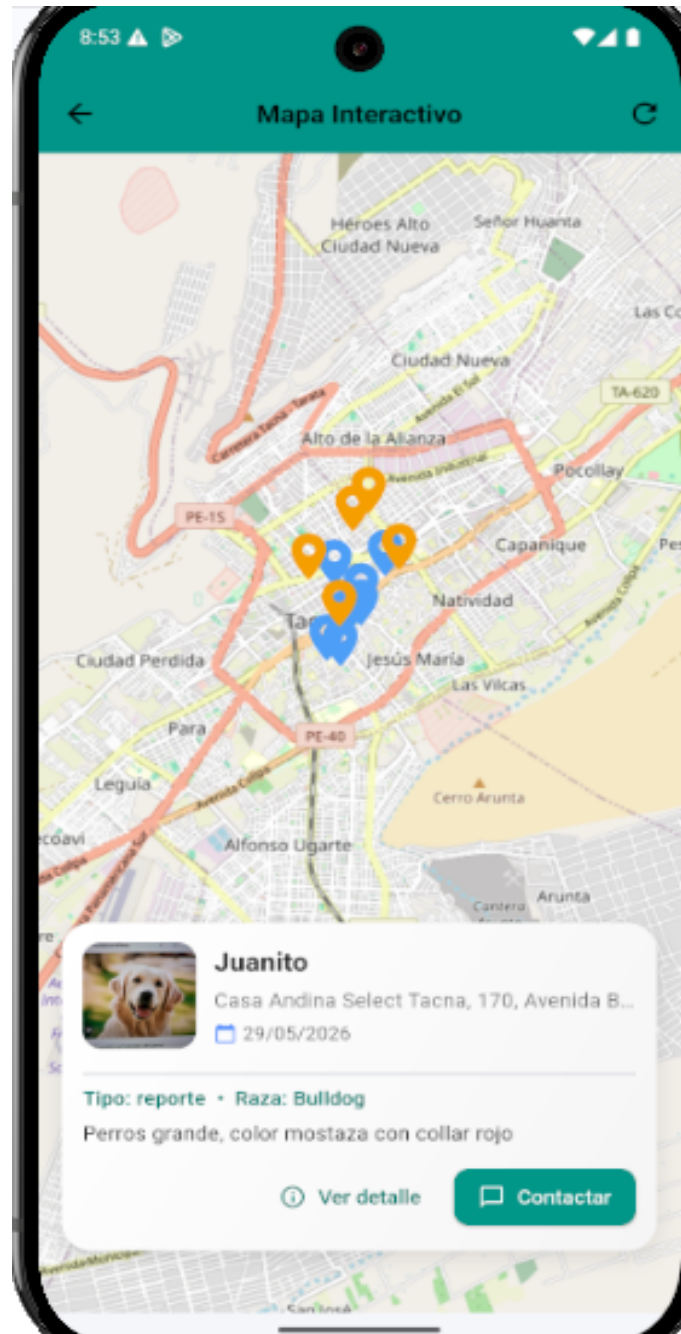


Figura 3.10.2: Captura de Pantalla - Ficha Modal Emergente al Pulsar un Marcador de Avistamiento.

 **Registrar Avistamiento**

 **Foto del avistamiento**



Agregar foto del avistamiento
Toca para seleccionar desde galería o cámara

 **Cámara**

 **Galería**

 **Ubicación del avistamiento**

Seleccionar desde el mapa 

 **Fecha y hora del avistamiento**

Fecha 

Hora 

3.11 RF-011: Sistema de Notificaciones Push y Alertas Comunitarias (FCM)

Descripción

Módulo de infraestructura que canaliza alertas prioritarias mediante Firebase Cloud Messaging y el protocolo HTTP v1:

- Notificaciones a toda la comunidad cuando se reporta una mascota perdida en su distrito.
- Notificaciones directas y privadas al dueño cuando se registra un avistamiento coincidente.
- Notificaciones locales en la barra de estado de Android con sonido y vibración personalizadas.

Objetivos

- Movilizar a la comunidad en los primeros 60 minutos del extravío (ventana crítica).
- Proveer un centro de notificaciones donde consultar el historial de avisos.

Caso de Uso CU-011: Gestión y Despacho de Notificaciones Push

Parámetro	Especificación
Actor Principal	Sistema Automático / Firebase Cloud Messaging
Descripción	Emisión y recepción de notificaciones en primer y segundo plano.
Precondición	Permisos de notificación concedidos y token FCM registrado en Firestore.
Poscondición	Se entrega la alerta visual y sonora en el dispositivo del usuario.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	Un usuario registra un nuevo reporte o avistamiento coincidente.	NotificacionServicio construye la carga JSON con título, cuerpo y metadatos.
2		Se emite la petición al backend de FCM HTTP v1.

Paso	Acción del Actor	Respuesta del Sistema
3		El servidor de Google despacha la notificación a los dispositivos suscritos.
4	El usuario recibe la notificación en la barra de estado del teléfono.	El sistema genera el aviso sonoro y vibratorio.
5	El usuario pulsa la notificación.	El aplicativo se abre y redirige directamente a la pantalla de la mascota en cuestión.

Flujo de Excepciones

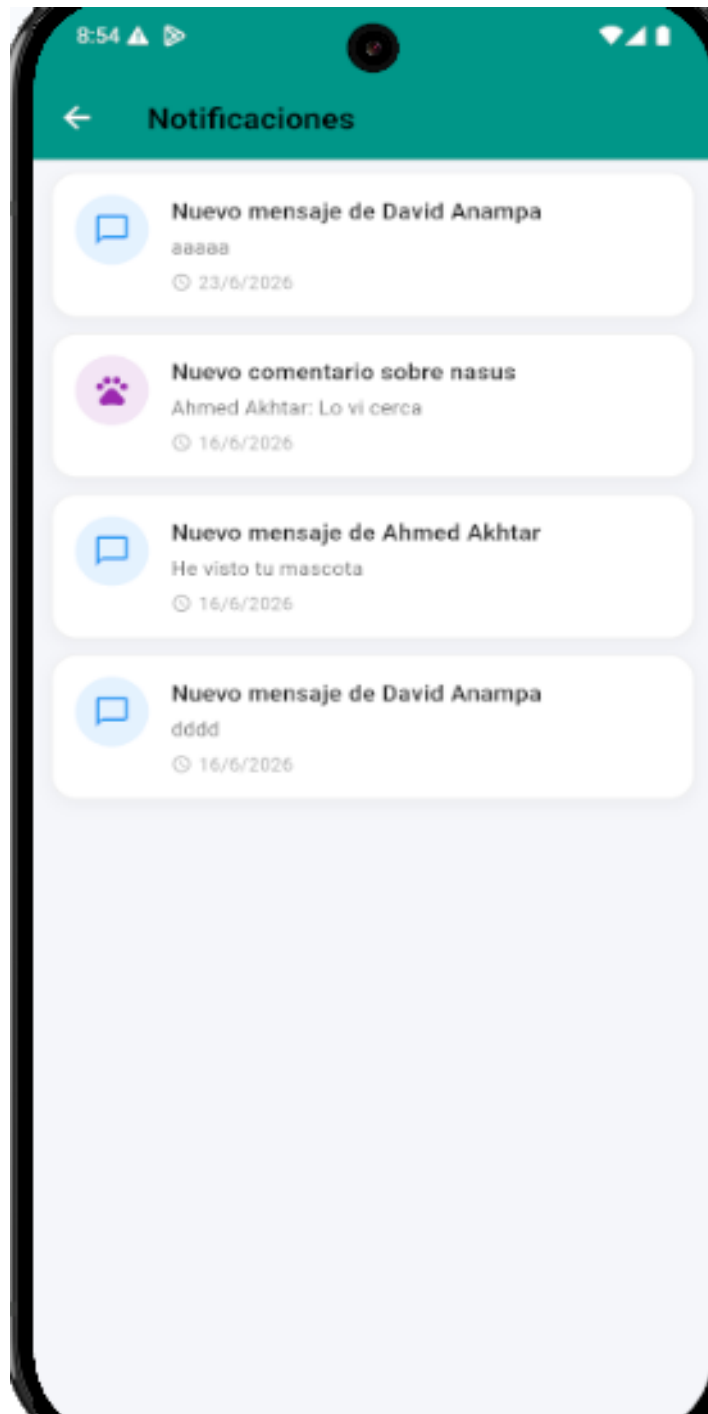
- **E1 - Permisos denegados:** Si el usuario no otorgó permisos, las notificaciones se almacenan únicamente en el centro de notificaciones de la app sin generar alerta en el sistema operativo.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/usuario/pantalla_notificacion.dart.
- **Controles Visuales:**
 - Lista Cronológica de Notificaciones: Con distinción visual entre leídas y no leídas.
 - Botón "Marcar todo como leído": Actualiza masivamente el estado en Firestore.

Evidencias Técnicas Requeridas

Figura 3.11.1: *Captura de Pantalla - Centro de Notificaciones dentro del Aplicativo Móvil.*



3.12 RF-012: Sistema de Comentarios, Reacciones y Respuestas en Hilos

Descripción

Canal comunitario dentro de cada reporte de mascota donde los voluntarios pueden aportar datos de avistamiento, fotografías secundarias, reaccionar con Likes o Dislikes para validar la veracidad de la información y responder a comentarios específicos organizados en árboles jerárquicos (Replies).

Objetivos

- Fomentar la inteligencia colectiva y colaboración ciudadana en tiempo real.
- Validar colaborativamente qué pistas sobre el paradero del animal son certeras.

Caso de Uso CU-012: Comentar e Interactuar en Reportes

Parámetro	Especificación
Actor Principal	Voluntario / Usuario Comunitario
Descripción	Publicación de comentarios, reacciones y respuestas anidadas en un reporte.
Precondición	Sesión de usuario activa y reporte de mascota existente.
Poscondición	Se actualiza la subcolección de comentarios y los contadores de reacciones.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario abre la ficha de una mascota y presiona el icono de comentarios.	El sistema abre <code>PantallaComentarios</code> listando los aportes en tiempo real.
2	El usuario redacta un mensaje (ej. "Vi un perro idéntico por la Av. Bolognesi hace una hora").	El sistema valida que el campo de texto contenga información válida.
3	El usuario presiona el botón de enviar.	<code>ComentariosViewModel</code> añade el documento en la subcolección <code>comentarios</code> .

Paso	Acción del Actor	Respuesta del Sistema
4	Otro usuario lee el comentario y presiona el botón "Me Gusta".	El sistema incrementa atómicamente el contador de likes en Firestore.
5	Un usuario presiona "Responder" en ese comentario.	El sistema abre <code>RepliesPage</code> permitiendo continuar el hilo de discusión específico.

Flujo de Excepciones

- **E1 - Comentario vacío:** Se inhibe el botón de envío si el campo no contiene caracteres alfanuméricos.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/usuario/pantalla_comentarios.dart`, `lib/vista/usuario/replies_page.dart`.
- **Controles Visuales:**
 - Lista de Comentarios: Con avatar, nombre del autor, fecha y texto.
 - Botones de Reacción: Contador dinámico de Likes e icono de respuesta.
 - Campo de Entrada Inferior: Barra fija con campo de texto y botón de envío.

Evidencias Técnicas Requeridas

Figura 3.12.1: *Captura de Pantalla - Módulo Comunitario de Comentarios y Reacciones.*

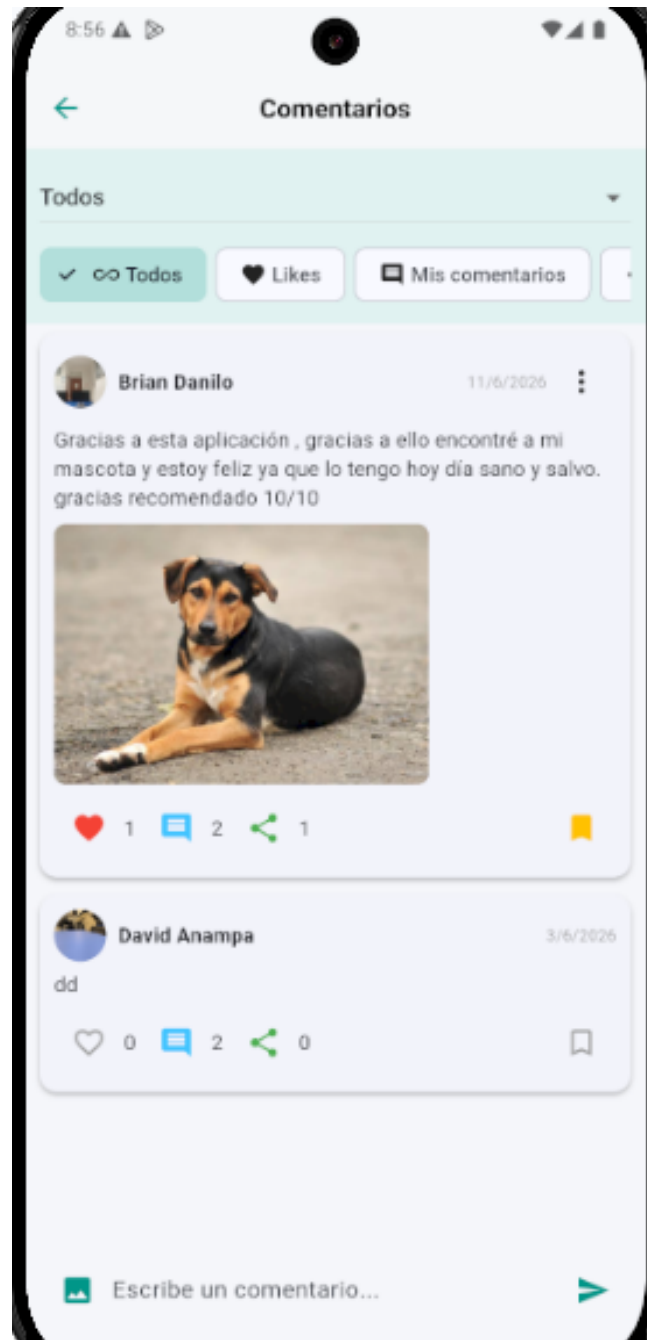
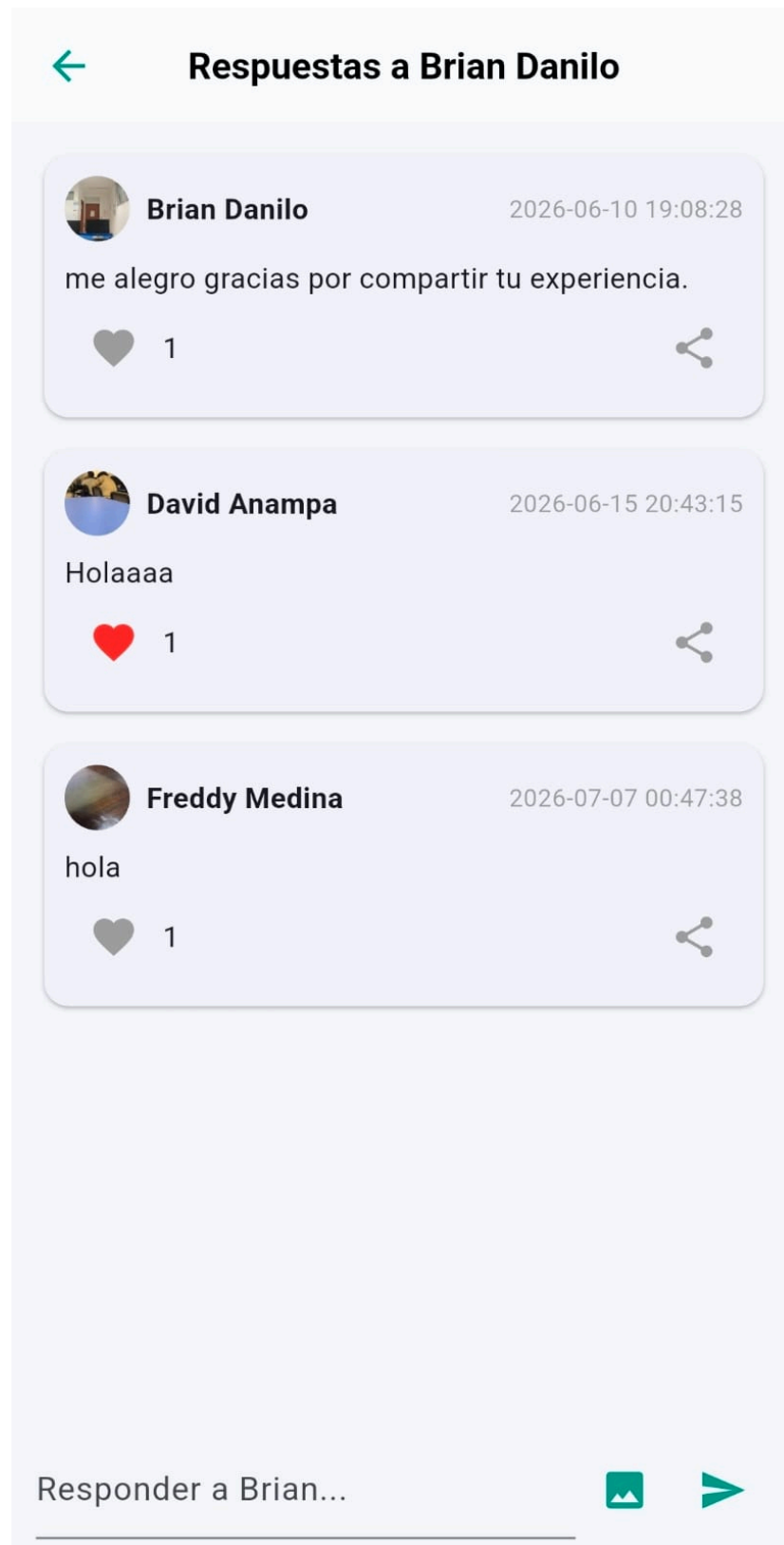


Figura 3.12.2: Captura de Pantalla - Hilo de Respuestas Anidadas (*RepliesPage*).

3.13 RF-013: Mensajería Privada y Chat en Tiempo Real entre Usuarios

Descripción

Provee una sala de chat privada y cifrada entre el voluntario que avistó al animal y el dueño de la mascota. Permite coordinar el rescate compartiendo detalles precisos y mensajes de texto instantáneos sincronizados mediante colecciones en Cloud Firestore.

Objetivos

- Preservar la privacidad evitando la exposición pública de números de teléfono personales.
- Agilizar la coordinación directa para la recuperación del animal.

Caso de Uso CU-013: Chatear en Privado con el Informante

Parámetro	Especificación
Actor Principal	Voluntario y Dueño de la Mascota
Descripción	Comunicación bidireccional instantánea mediante mensajería en tiempo real.
Precondición	Ambos usuarios deben contar con cuentas activas.
Poscondición	Los mensajes se sincronizan y persisten en la sala de chat correspondiente.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El voluntario presiona "Contactar Dueño" desde la ficha de la mascota.	El sistema inicializa o recupera el canal de chat (<code>chats/{chatId}</code>).
2	Se despliega <code>PantallaChat</code> mostrando el historial de la conversación.	El sistema suscribe un <code>StreamBuilder</code> a la subcolección de mensajes.
3	El usuario redacta un mensaje de texto y presiona el botón de envío.	El sistema escribe el mensaje en Firestore con timestamp del servidor.

Paso	Acción del Actor	Respuesta del Sistema
4	El receptor visualiza el mensaje entrante en su pantalla en milisegundos.	El sistema actualiza el estado de entrega del mensaje.

Flujo de Excepciones

- **E1 - Cuenta del interlocutor suspendida:** Se inhabilita el campo de texto y se muestra: "No es posible enviar mensajes a este usuario debido a que su cuenta se encuentra inactiva".

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/chat/pantalla_chat.dart`.
- **Controles Visuales:**
 - Cabecera de Chat: Con foto de perfil, nombre del contacto y estado en línea.
 - Burbujas de Mensaje: Mensajes enviados (alineados a la derecha en azul) y recibidos (a la izquierda en gris).
 - Campo Inferior de Entrada: Con soporte para texto y envío instantáneo.

Evidencias Técnicas Requeridas

Figura 3.13.1: *Captura de Pantalla - Sala de Chat Privada entre Voluntario y Dueño.*

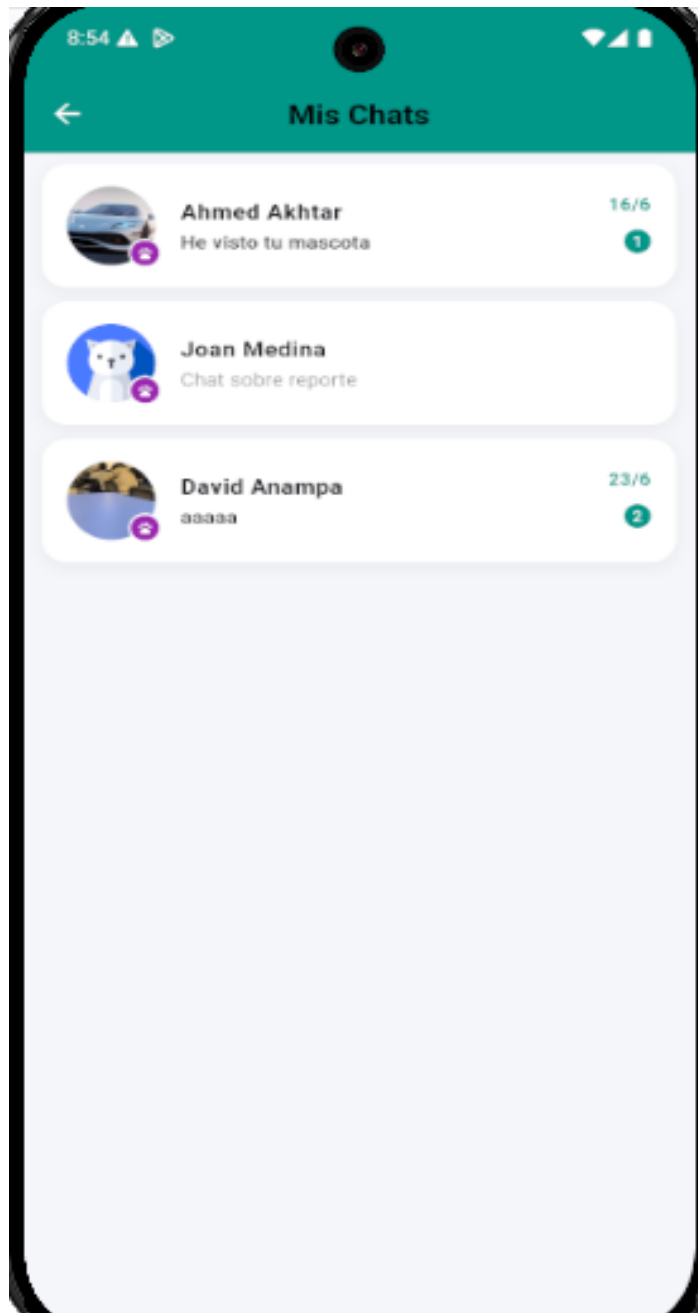
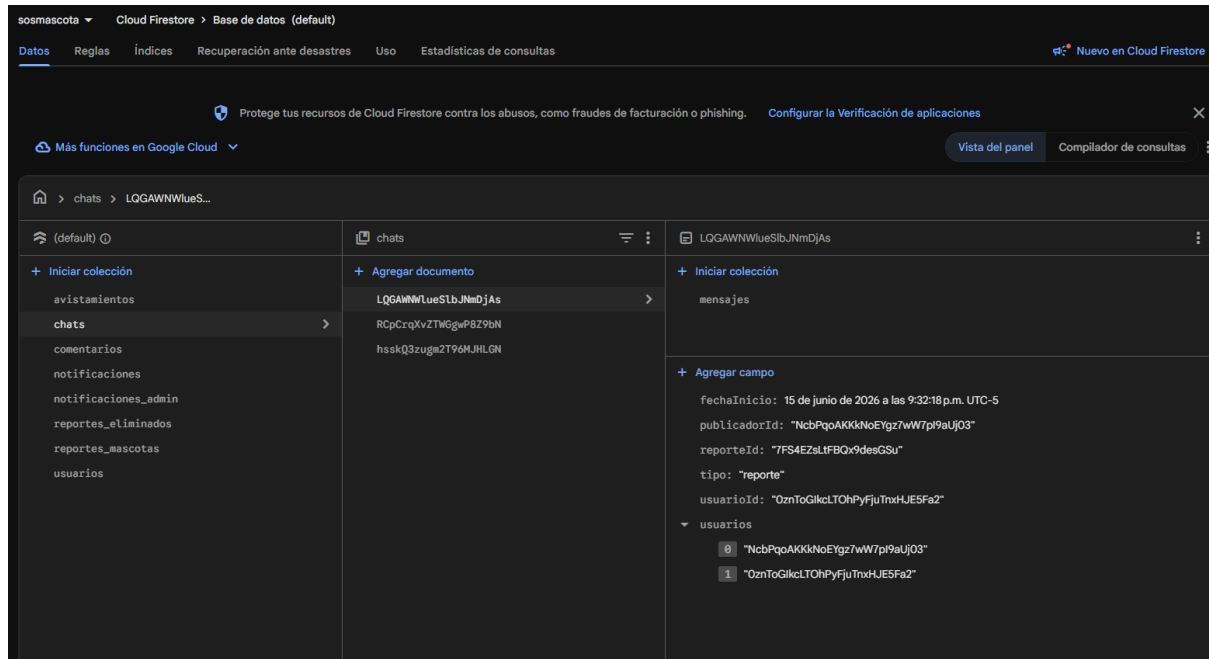


Figura 3.13.2: Evidencia de Base de Datos - Colección *chats* y Subcolección *mensajes* en *Firestore*.



3.14 RF-014: Generación y Exportación de Cartel de Búsqueda en Formato PDF

Descripción

Módulo de renderizado vectorial (pdf y printing) que compila en milisegundos un afiche formal de "SE BUSCA":

- Encabezado con tipografía de alto impacto en color rojo.
- Fotografía principal en alta resolución con marco destacado.
- Datos descriptivos (nombre, raza, edad, señas particulares).
- Número de contacto del dueño y recompensa económica ofrecida en Soles.
- Botón directo para compartir el archivo .pdf por WhatsApp, correo o imprimirlo físicamente.

Objetivos

- Proveer material físico imprimible para pegado en postes y comercios de Tacna.
- Facilitar la viralización del afiche digital en redes sociales.

Caso de Uso CU-014: Generar Afiche Imprimible en PDF

Parámetro	Especificación
Actor Principal	Dueño de Mascota
Descripción	Renderizado vectorial y exportación de afiche de búsqueda en formato estándar PDF.
Precondición	Reporte con fotografía y datos descriptivos completos.
Poscondición	Se compila el archivo PDF y se abre el gestor de impresión del dispositivo.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El dueño ingresa a la ficha de su mascota y presiona "Descargar Cartel PDF".	El sistema activa el motor de renderizado vectorial de documentos PDF.

Paso	Acción del Actor	Respuesta del Sistema
2		El sistema descarga temporalmente la fotografía de alta calidad desde Firebase Storage.
3		Compila el documento PDF estructurando título, fotografía, datos, mapa de referencia y contacto.
4		Abre la ventana nativa de previsualización e impresión de Android/iOS.
5	El usuario presiona "Imprimir" o "Compartir archivo".	El sistema deriva el archivo al canal seleccionado por el usuario.

Flujo de Excepciones

- **E1 - Falla de red al descargar foto:** Se utiliza una imagen de respaldo institucional y se advierte al usuario.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/reportes/pantalla_ver_reportes.dart (módulo de impresión).
- **Controles Visuales:**
 - Botón Flotante "Generar Cartel PDF": Con icono de documento de impresión.
 - Visor Nativo de PDF: Con opciones de zoom, selección de páginas e impresión directa.

Evidencias Técnicas Requeridas

Figura 3.14.1: Captura de Pantalla - Afiche Vectorial de Búsqueda de Mascota en Formato PDF.

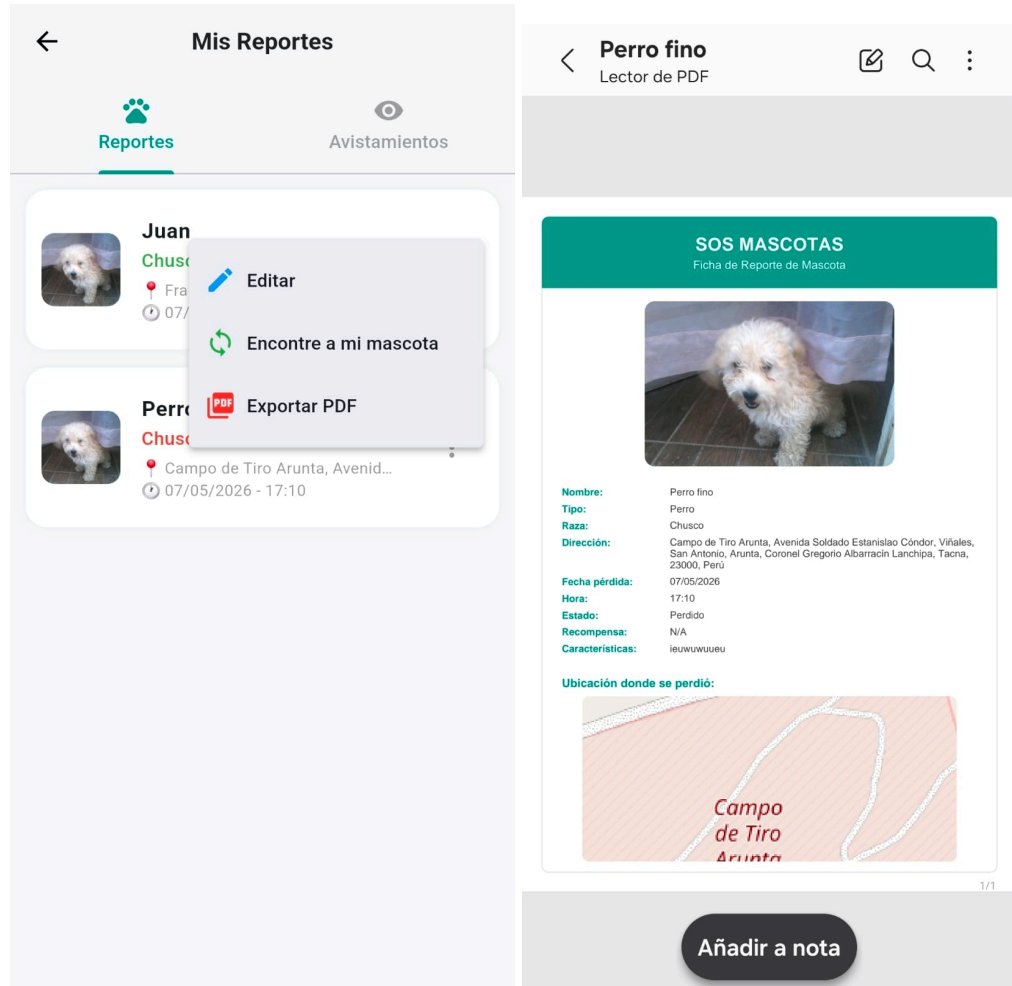
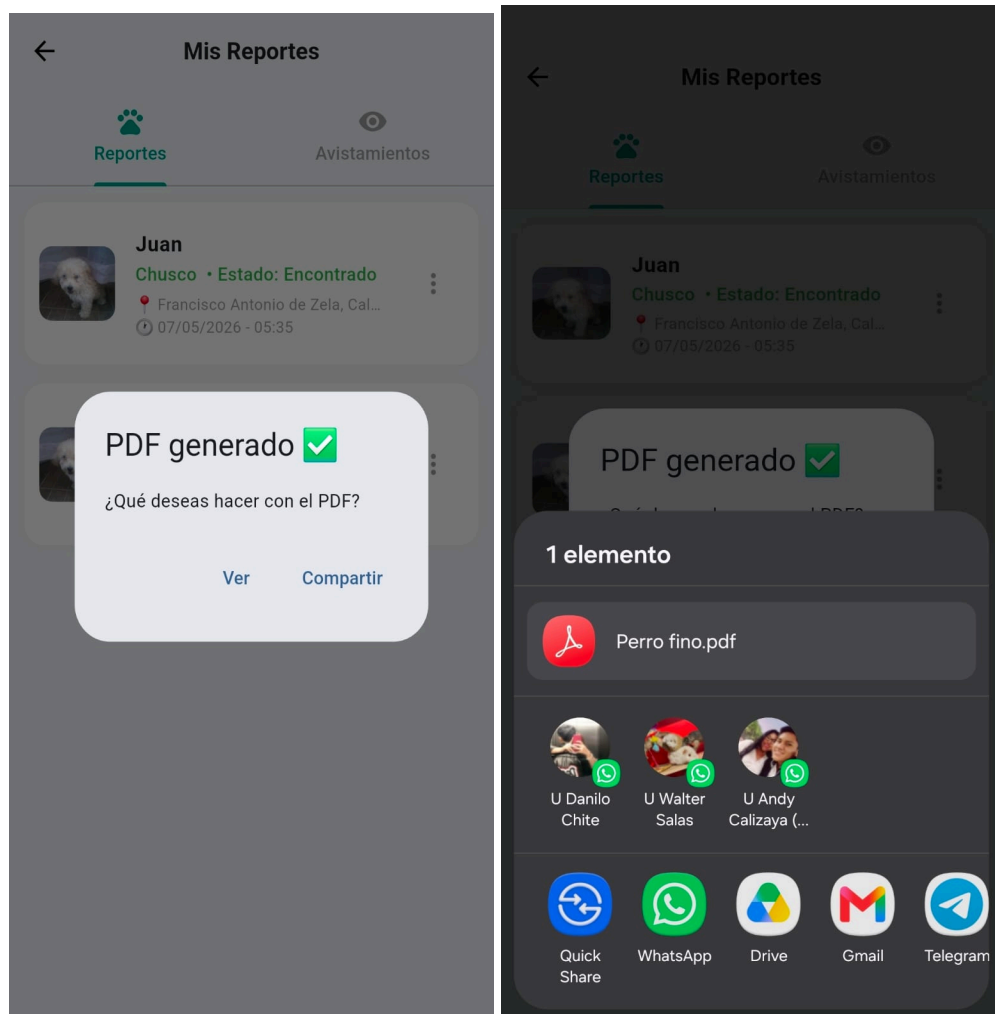


Figura 3.14.2: Captura de Pantalla - Menú Nativo de Impresión y Exportación a Redes Sociales.



3.15 RF-015: Gestión de Perfil de Usuario

Descripción

Permite a cada voluntario visualizar sus datos personales.

Objetivos

- Gamificar la participación ciudadana reconociendo el esfuerzo de los voluntarios de Tacna.
- Proveer gestión de datos de contacto (teléfono y fotografía de perfil).

Caso de Uso CU-015: Consultar Perfil

Parámetro	Especificación
Actor Principal	Voluntario / Usuario Registrado
Descripción	Consulta de datos personales, historial
Precondición	Sesión de usuario activa.
Poscondición	La vista despliega el balance actualizado de puntos y opciones de edición.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El usuario selecciona la pestaña "Perfil" en la barra inferior de navegación.	El sistema abre PantallaPerfil consultando los datos en tiempo real de Firestore.
2		Se renderiza el avatar, nombre oficial verificado por RENIEC, correo y teléfono.
3		Se muestra la tarjeta destacada con el saldo (+10 por avistamiento, +50 por reporte).
4	El usuario presiona "Editar Perfil".	El sistema permite actualizar su número de teléfono o fotografía de perfil.
5	El usuario presiona "Guardar Cambios".	PerfilVM persiste las modificaciones en Firestore y muestra alerta de éxito.

Flujo de Excepciones

- **E1 - Intento de alterar Nombres:** Los campos de identidad se mantienen deshabilitados (inmutables) para salvaguardar la veracidad ciudadana.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/usuario/pantalla_perfil.dart`.
- **Controles Visuales:**
 - Formulario de Datos: Con campos bloqueados para nombres, y campo editable para celular.
 - Botón de Cerrar Sesión: Invoca a `FirebaseAuth.signOut()` retornando a login.

Evidencias Técnicas Requeridas

Figura 3.15.1: *Captura de Pantalla - Perfil del Voluntario*

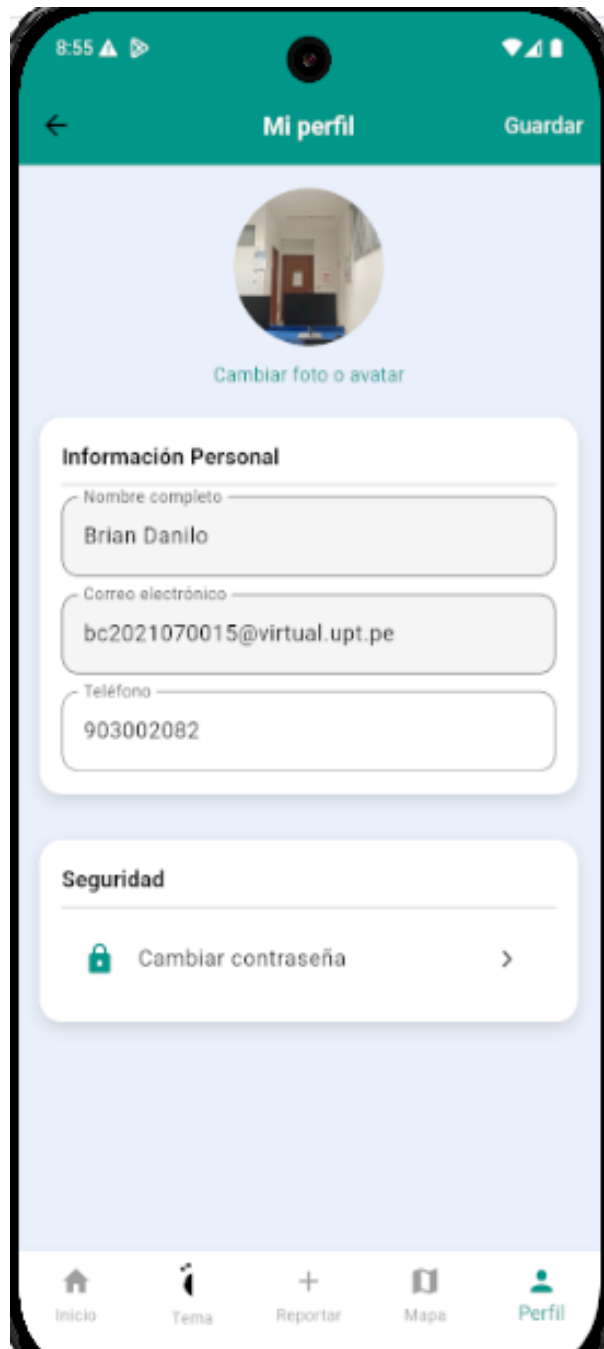
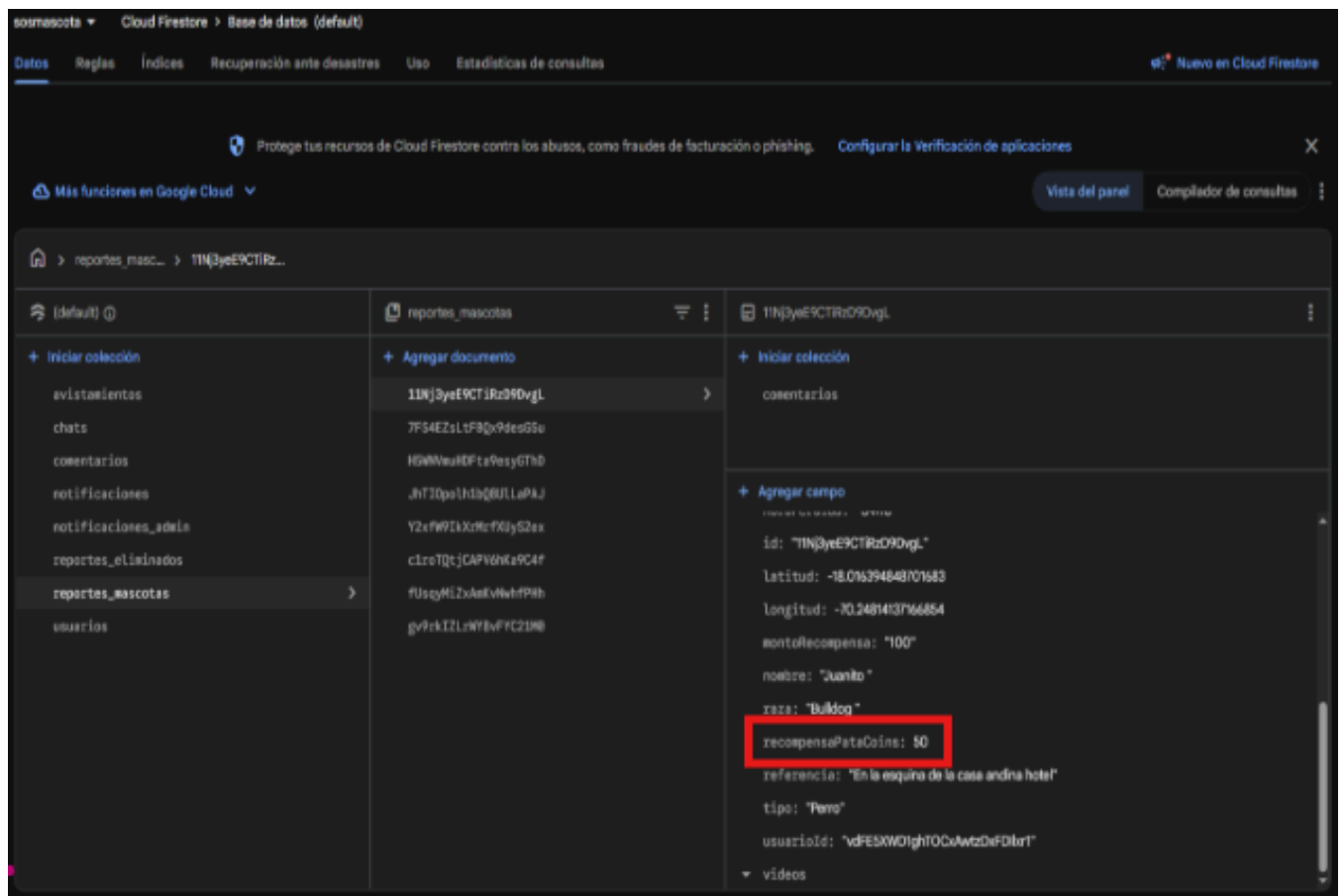


Figura 3.15.2: Evidencia de Base de Datos - Saldo de Almacenado en Firestore.

3.16 RF-016: Panel de Administración, Moderación de Usuarios y Control de Roles

Descripción

Panel de acceso restringido para administradores de SOS Mascota que permite:

- Listar todos los usuarios con paginación y búsqueda por correo.
- Modificar roles de usuario (usuario a admin o viceversa).
- Activar o suspender cuentas de usuarios de forma inmediata (bloqueo reactivo en tiempo real con `auth_wrapper.dart`).
- Historial de auditoría de usuarios eliminados y colaboradores desvinculados.

Objetivos

- Mantener la seguridad y moderar conductas inapropiadas o reportes fraudulentos.
- Controlar los permisos de administración del sistema.

Caso de Uso CU-016: Moderar Usuarios y Gestionar Roles

Parámetro	Especificación
Actor Principal	Administrador del Sistema
Descripción	Gestión de estados de cuenta (activación/bloqueo) y asignación de privilegios de rol.
Precondición	El actor debe contar con rol admin verificado en Firestore.
Poscondición	Las modificaciones en el documento del usuario se propagan reactivamente.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El administrador inicia sesión y es dirigido a <code>PantallaInicioAdmin</code> .	El sistema habilita los módulos de control administrativo.
2	Selecciona la opción "Gestión de Usuarios".	El sistema abre <code>PantallaAdminUsuarios</code> listando todas las cuentas registradas.

Paso	Acción del Actor	Respuesta del Sistema
3	Digita en el buscador.	El sistema filtra en tiempo real la lista de usuarios.
4	El administrador pulsa el conmutador de estado para suspender a un usuario infractor.	El sistema actualiza el campo <code>activo = false</code> en Firestore.
5		Si el usuario está conectado en su dispositivo, <code>auth_wrapper.dart</code> lo expulsa de inmediato.
6	El administrador pulsa "Cambiar Rol" para promover a un voluntario a administrador.	El sistema actualiza el campo <code>rol = 'admin'</code> confirmando la operación.

Flujo de Excepciones

- **E1 - Intento de auto-bloqueo:** El sistema impide que un administrador desactive su propia cuenta para prevenir bloqueos accidentales del panel.

Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** `lib/vista/admin/pantalla_adminusuarios.dart`.
- **Controles Visuales:**
 - Barra de Filtrado Superior: Búsqueda por correo o nombre.
 - Tarjetas de Usuario: Con distintivo de rol (`admin/usuario`) e interruptor (`Switch`) de estado activo.
 - Diálogo Modal de Confirmación: Previene suspensiones accidentales.

Evidencias Técnicas Requeridas

Figura 3.16.1: *Captura de Pantalla - Panel Administrativo de Moderación de Usuarios.*

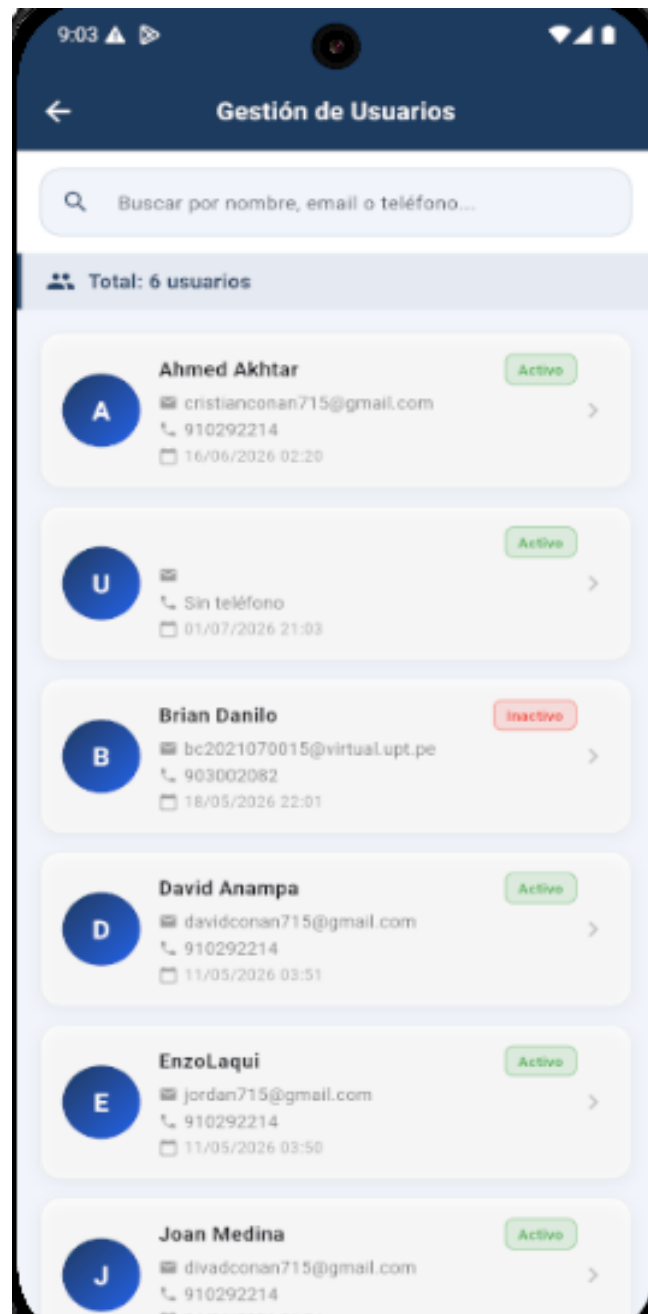
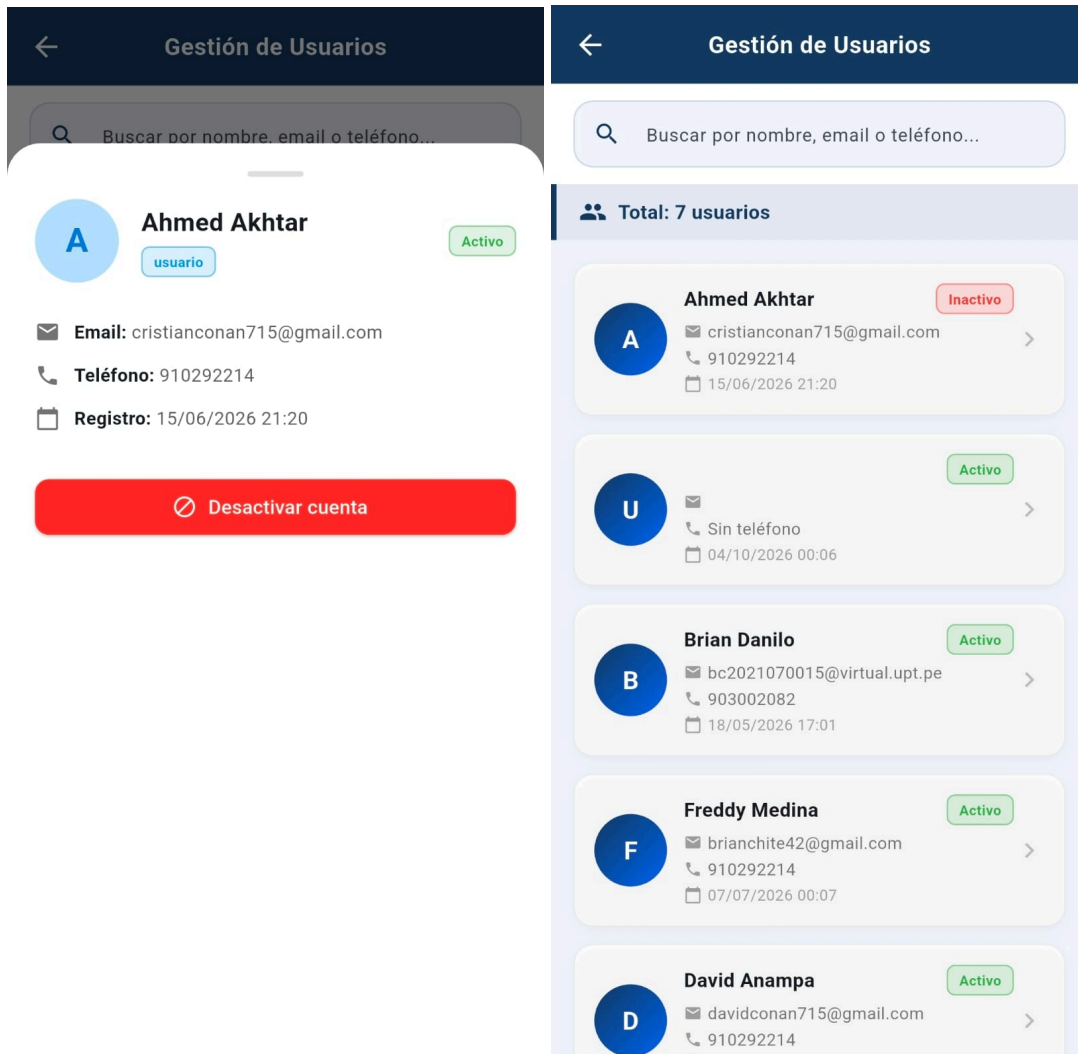


Figura 3.16.2: *Captura de Pantalla - Alerta de Expulsión en el Dispositivo del Usuario Bloqueado.*



3.17 RF-017: Dashboard Estadístico y Métricas Generales del Sistema

Descripción

Módulo analítico institucional (AdminEstadisticasVM) que genera gráficos estadísticos dinámicos con métricas clave para la toma de decisiones:

- Total de mascotas perdidas vs. mascotas encontradas (Tasa de efectividad comunitaria).
- Distribución porcentual de extravíos por distrito de Tacna (Tacna Centro, Gregorio Albarracín, Alto de la Alianza, Pocollay, Ciudad Nueva).
- Distribución por especie (caninos vs felinos).
- Promedio de avistamientos registrados por semana y análisis de tendencias.

Objetivos

- Medir el impacto social y efectividad del sistema SOS Mascota en la región de Tacna.
- Identificar distritos con mayor índice de vulnerabilidad para enfocar campañas de rescate.

Caso de Uso CU-017: Consultar Métricas y Dashboard Estadístico

Parámetro	Especificación
Actor Principal	Administrador del Sistema
Descripción	Generación y visualización de gráficos estadísticos agregados del sistema.
Precondición	Cuenta con privilegios de rol <code>admin</code> .
Poscondición	La vista renderiza tarjetas de indicadores y gráficos de barras/pastel interactivos.

Narrativa del Caso de Uso

Paso	Acción del Actor	Respuesta del Sistema
1	El administrador selecciona la pestaña "Estadísticas" en el panel administrativo.	El sistema abre PantallaAdminEstadisticas.
2		AdminEstadisticasVM ejecuta consultas de agregación sobre las colecciones reportes y avistamientos.
3		Se calculan los porcentajes de resolución, casos activos y total de animales asistidos.
4		Se renderizan gráficos circulares por especie y gráficos de barras por distrito de Tacna.
5	El administrador toca un segmento del gráfico para ver detalles cuantitativos.	El sistema despliega el tooltip interactivo con el conteo exacto.

Flujo de Excepciones

- **E1 - Sin datos registrados:** Los gráficos muestran un estado informativo neutral con recomendación de ingreso de reportes.

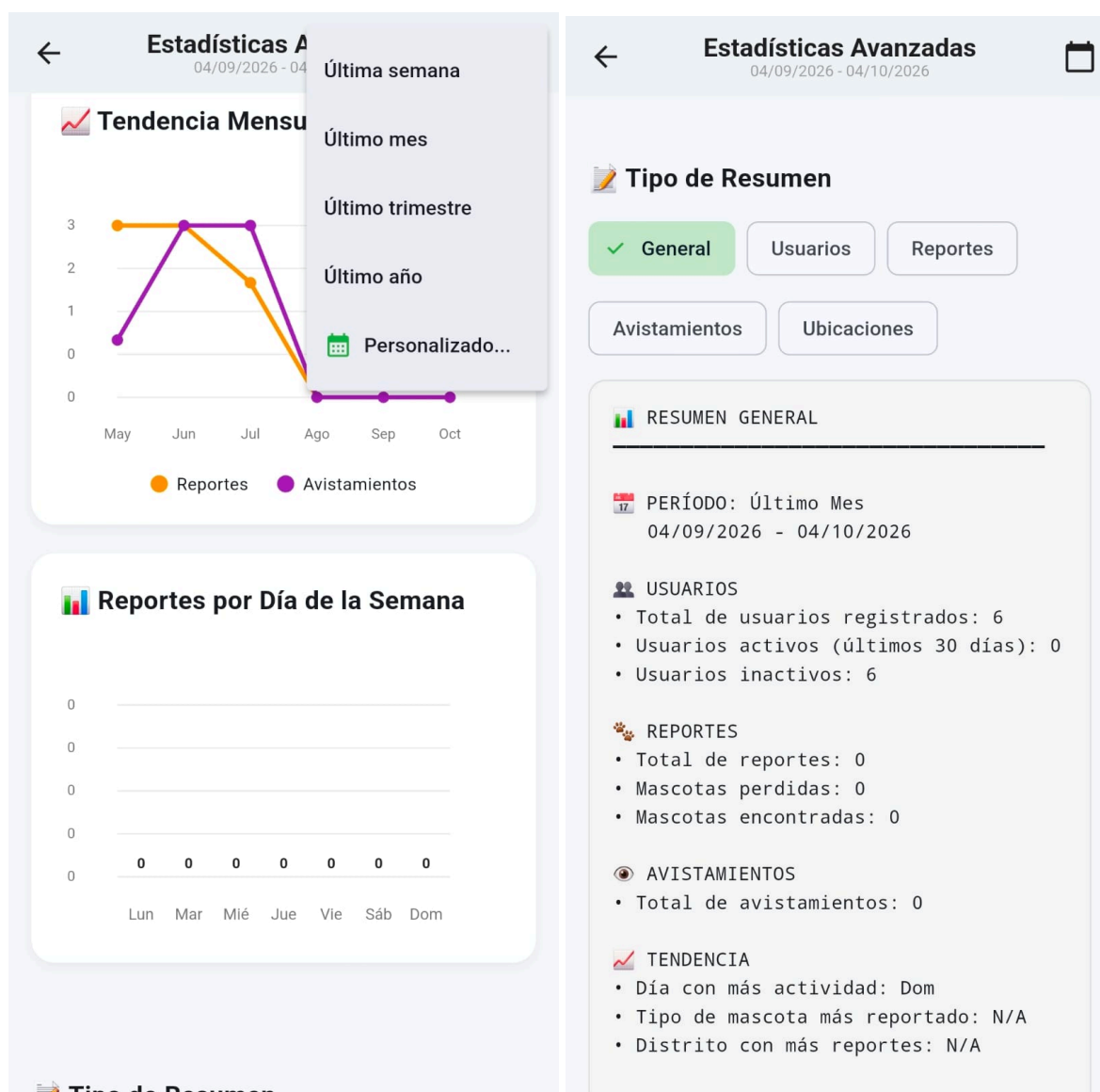
Especificación de Interfaz y Controles de Pantalla

- **Ruta de Código:** lib/vista/admin/PantallaAdminEstadisticas.dart.
- **Controles Visuales:**
 - Tarjetas de Resumen KPI: "Total Reportes", "Mascotas Encontradas", "Avistamientos".
 - Gráfico Circular (Pie Chart): Distribución porcentual por especie de animal.
 - Gráfico de Barras (Bar Chart): Incidentes agrupados por distrito de la ciudad de Tacna.

Evidencias Técnicas Requeridas

Figura 3.17.1: *Captura de Pantalla - Dashboard Estadístico de Métricas de Mascotas en Tacna.*



Figura 3.17.2: Captura de Pantalla - Gráficos Comparativos por Especie y por Mes.

4. Planificación Cronológica de Tareas (Metodología SCRUM)

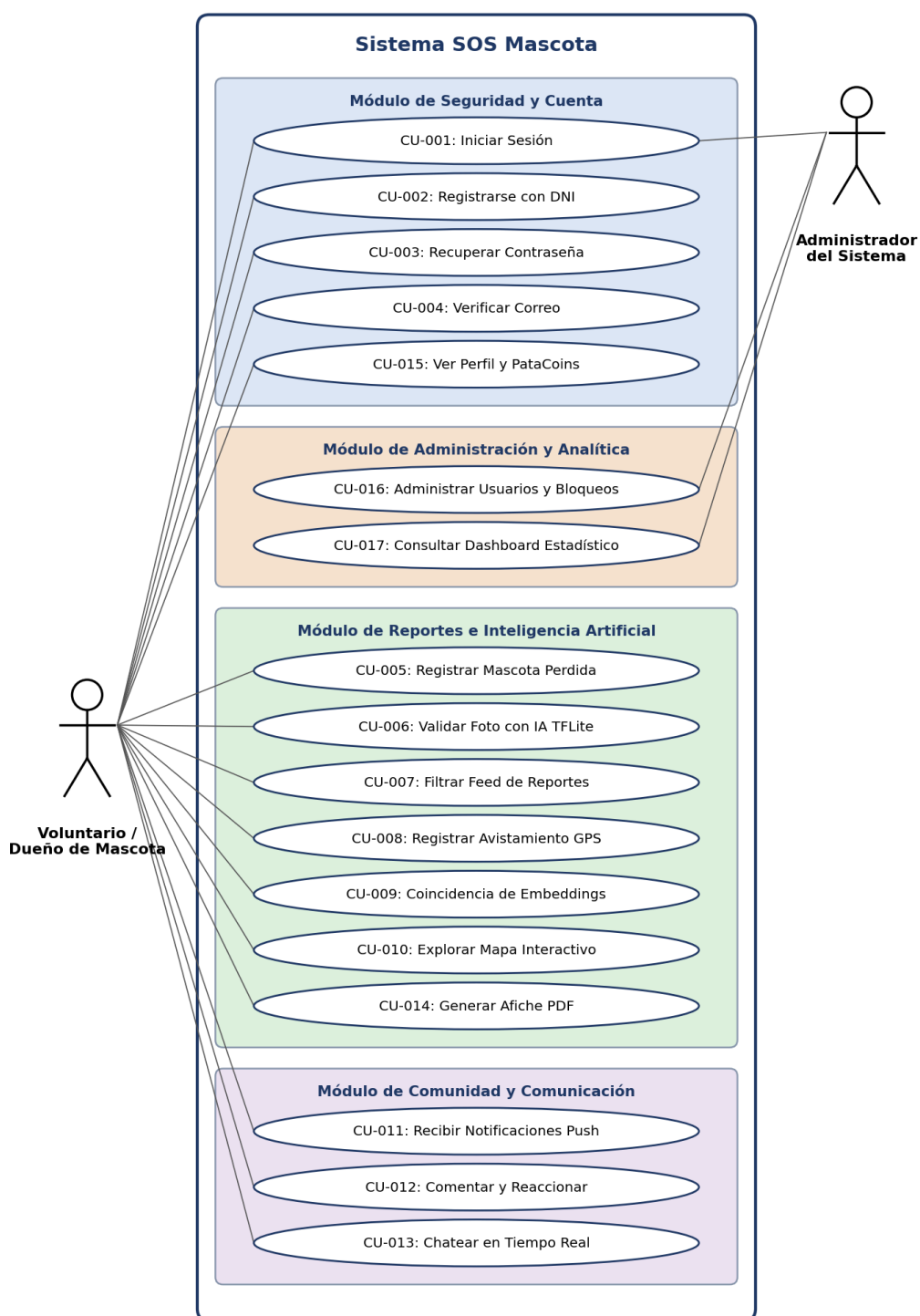
La siguiente tabla establece la distribución temporal de actividades correspondiente a la iteración evaluada:

Caso de Uso / Tarea	Nombre de la Tarea Técnica	Fecha Inicio	Fecha Fin	Responsable
CU01 (RF-001)	Desarrollo de Autenticación Segura y Wrapper de Sesión en Firebase	16/09/2026	19/09/2026	Equipo Backend / Auth
CU02 (RF-002)	Integración Formulario de Registro	20/09/2026	23/09/2026	Fullstack Developer
CU03 (RF-003/004)	Recuperación de Contraseña y Verificación de Correo	24/09/2026	26/09/2026	Backend Lead
CU05 (RF-005)	Maquetación y Lógica del Formulario Wizard de Reporte en 3 Pasos	27/09/2026	01/10/2026	Frontend Lead
CU06 (RF-006)	Implementación de Inferencia de IA On-Device con TensorFlow Lite	02/10/2026	05/10/2026	AI / Mobile Engineer
CU07 (RF-007)	Feed Dinámico de Reportes con Filtros y Tarjetas Interactivas	06/10/2026	09/10/2026	Frontend Lead
CU08 (RF-008)	Módulo de Avistamiento Rápido con Cámara y Coordenadas GPS	10/10/2026	13/10/2026	Mobile Developer
CU09 (RF-009)	Algoritmo de Cotejo Biométrico y Similitud Coseno de Embeddings	14/10/2026	17/10/2026	AI / Backend Lead

Caso de Uso / Tarea	Nombre de la Tarea Técnica	Fecha Inicio	Fecha Fin	Responsable
CU10 (RF-010)	Cartografía Digital con FlutterMap y Marcadores OpenStreetMap	18/10/2026	21/10/2026	Mobile Developer
CU11 (RF-011)	Dispersión de Notificaciones Push con FCM v1 y Handlers de Background	22/10/2026	24/10/2026	Backend Lead
CU12 (RF-012/013)	Interacción Comunitaria: Comentarios, Hilos de Respuestas y Chat	25/10/2026	28/10/2026	Frontend / Mobile Dev
CU14 (RF-014)	Motor de Generación y Maquetación de Afiches de Búsqueda PDF	29/10/2026	31/10/2026	Fullstack Developer
CU15 (RF-015/016)	Panel de Perfil, y Moderación de Usuarios Administrador	01/11/2026	04/11/2026	Fullstack Developer
CU17 (RF-017)	Dashboard Analítico con Gráficos Estadísticos y Métricas por Distrito	05/11/2026	08/11/2026	Fullstack Developer
QA / AUDITORÍA	Auditoría de Código (9 Reglas de Código Limpio) y Suite de Pruebas	09/11/2026	12/11/2026	Auditor Senior de QA

5. Casos de Uso y Escenarios del Sistema

El siguiente diagrama general articula los 17 requerimientos entre los dos roles del sistema:



6. Recursos de Software y Hardware

Tabla 01: Recursos de Software

Componente	Tipo de Recurso	Detalle de Especificación Técnica
Lenguaje de Programación	Lenguaje Tipado con Null Safety	Dart SDK ^3.8.1
Framework de Desarrollo	Multiplataforma Híbrida Móvil	Flutter Framework 3.29+ (Android / iOS / Web)
Base de Datos Principal	NoSQL Distribuida en Tiempo Real	Google Cloud Firestore
Servicio de Autenticación	Gestión de Identidad y Acceso (IAM)	Firebase Authentication SDK
Almacenamiento de Medios	Repositorio de Archivos y Fotos	Firebase Cloud Storage
Motor de Visión Artificial	Redes Neuronales On-Device	TensorFlow Lite Flutter (tflite_flutter)
Servicio Cartográfico	Mapas y Tiles Libres	OpenStreetMap API + flutter_map
Servicio de Notificaciones	Mensajería Push Global	Firebase Cloud Messaging (FCM HTTP v1)
Entornos de Desarrollo	IDEs de Compilación y Depuración	Visual Studio Code / Android Studio Iguana
Control de Versiones	Repositorio Distribuido en la Nube	Git + GitHub
Dispositivo de Simulación	Emulador / Hardware Físico	Android Virtual Device (AVD) / Smartphone Físico Android 13

Tabla 02: Recursos de Hardware

Componente de Hardware	Requisito Mínimo del Entorno	Requisito Recomendado para Compilación
Memoria RAM	8 GB DDR4	16 GB DDR4/DDR5
Disco Duro / Almacenamiento	256 GB SSD SATA	512 GB SSD NVMe M.2
Procesador (CPU)	Intel Core i5 / AMD Ryzen 5 (2.5 GHz)	Intel Core i7 / AMD Ryzen 7 (3.5 GHz octa-core)
Cámara y Sensores Móviles	Sensor de Cámara 8 MP + GPS básico	Cámara 48 MP + A-GPS / GLONASS de alta precisión

7. Evaluación de la Iteración

Objetivos Alcanzados

1. Se implementaron, integraron y probaron con éxito los **17 requerimientos funcionales** del aplicativo móvil SOS Mascota.
2. Se resolvió la inferencia de inteligencia artificial directamente en el dispositivo móvil (Edge Computing) mediante TensorFlow Lite, logrando clasificar imágenes de animales y extraer vectores de características sin depender de servidores externos.
3. Se garantizó la trazabilidad y separación de responsabilidades mediante la arquitectura **MVVM (Model-View-ViewModel)** y el gestor de estados **Provider**.
4. Se llevó a cabo una auditoría técnica completa basada en 9 reglas de código limpio (DRY, Fail Fast, Sin números mágicos, inmutabilidad, etc.), corrigiendo 39 hallazgos críticos de diseño.
5. La suite de pruebas unitarias (**flutter test**) arroja un **100% de casos aprobados** sin regresiones funcionales ni errores de compilación.

Objetivos No Alcanzados

- La exportación automática de métricas hacia hojas de cálculo Excel (**.xlsx**) fue diferida para la siguiente fase de transición del proyecto, priorizándose la generación de afiches en PDF y el Dashboard reactivo en pantalla.
- La migración de las credenciales maestras de Firebase a Cloud Functions intermedias se programó como tarea de cierre para el entorno de producción en la nube.

Elementos Incluidos en la Línea Base

Controladores y ViewModels

- **lib/vistamodelo/auth/login_vm.dart**
- **lib/vistamodelo/auth/registro_vm.dart**
- **lib/vistamodelo/auth/recuperar_vm.dart**
- **lib/vistamodelo/reportes/reporte_vm.dart**
- **lib/vistamodelo/reportes/avistamiento_vm.dart**
- **lib/vistamodelo/comentarios/comentarios_viewmodel.dart**
- **lib/vistamodelo/notificacion/notificacion_vm.dart**

- `lib/vistamodelo/usuario/perfil_vm.dart`
- `lib/vistamodelo/admin/admin_vm.dart`
- `lib/vistamodelo/admin/adminusuario_vm.dart`
- `lib/vistamodelo/admin/AdminReportes_vm.dart`
- `lib/vistamodelo/admin/AdminEstadisticas_vm.dart`

Vistas y Módulos de Usuario

- `lib/vista/auth/pantalla_login.dart`
- `lib/vista/auth/pantalla_registro.dart`
- `lib/vista/auth/pantalla_recuperar.dart`
- `lib/vista/auth/pantalla_verifica_email.dart`
- `lib/vista/usuario/pantalla_inicio.dart`
- `lib/vista/usuario/pantalla_perfil.dart`
- `lib/vista/usuario/pantalla_notificacion.dart`
- `lib/vista/usuario/pantalla_comentarios.dart`
- `lib/vista/usuario/replies_page.dart`
- `lib/vista/reportes/pantalla_reporte_mascota.dart`
- `lib/vista/reportes/pantalla_avistamiento.dart`
- `lib/vista/reportes/pantalla_ver_reportes.dart`
- `lib/vista/reportes/pantalla_mis_reportes.dart`
- `lib/vista/mapa/pantalla_mapa_interactivo.dart`
- `lib/vista/chat/pantalla_chat.dart`

Vistas de Administración

- `lib/vista/admin/pantalla_inicio_admin.dart`
- `lib/vista/admin/pantalla_adminusuarios.dart`
- `lib/vista/admin/pantallaAdminReporte.dart`
- `lib/vista/admin/pantallaAdminNotificaciones.dart`
- `lib/vista/admin/PantallaAdminEstadisticas.dart`
- `lib/vista/admin/pantalla_perfil_admin.dart`

Servicios y Utilidades

- `lib/servicios/auth_servicio.dart`
- `lib/servicios/api_dni_servicio.dart`
- `lib/servicios/notificacion_servicio.dart`
- `lib/servicios/servicio_tflite.dart`
- `lib/utls/auth_wrapper.dart`
- `lib/utls/imagen_util.dart`
- `lib/app.dart` (AppConfig, NavigationService)

8. Conclusiones

1. Se concluyó exitosamente el Plan de Iteración de la fase de construcción del sistema SOS Mascota, logrando implementar de manera articulada los 17 requerimientos funcionales demandados por la comunidad de rescate de animales en Tacna.
2. La adopción del patrón arquitectónico MVVM desacopló limpiamente la lógica de negocio de los componentes visuales, permitiendo que las vistas respondan de manera fluida y reactiva a los cambios en Cloud Firestore sin congelar el hilo principal de renderizado.
3. El procesamiento de visión computacional mediante TensorFlow Lite demostró ser una solución altamente eficiente para el contexto móvil, garantizando que únicamente imágenes legítimas de mascotas entren al sistema sin consumir ancho de banda de red en el dispositivo del voluntario.
4. La aplicación rigurosa de principios de calidad (DRY, Fail Fast, inmutabilidad y ausencia de variables globales) garantizó un código fuente robusto, estructurado y alineado con los estándares exigidos en el curso de Construcción de Software II.

9. Plan de Próximas Iteraciones

Para las fases subsiguientes de transición y despliegue final a producción, se tienen programadas las siguientes actividades:

- **Fase 02 – Iteración 02:** Despliegue de Firebase Cloud Functions para aislar las credenciales maestras de FCM y automatizar el mantenimiento de reportes caducados.
- **Fase 02 – Iteración 03:** Implementación de pruebas automatizadas de widgets y pruebas de integración extremo a extremo (E2E) con Flutter Driver.
- **Fase 03 – Iteración 01:** Empaquetado final de producción (.apk y .aab) firmado con llaves criptográficas de lanzamiento y publicación en Google Play Store.

10. Estado del Repositorio y Trazabilidad

- **Organización / Propietario:** David-Anampa
- **Nombre del Repositorio:** CONSTRUCCION_SOFTWARE_II_ANAMPA_CHITE
- **URL** **Oficial:**
https://github.com/David-Anampa/CONSTRUCCION_SOFTWARE_II_ANAMPA_CHITE
- **Rama de Trabajo:** UNIDAD-I
- **Evidencias Técnicas Generadas:**
 - Informe de Auditoría de Código: reporte_auditoria_codigo.html
 - Informe en Texto Plano: reporte_auditoria_codigo.txt
 - Plan de Iteración Completo: PLAN_DE_ITERACION_SOS_MASCOTA.md
 - Informe de Laboratorio 01:
INFORME_LABORATORIO_01_REVISION_CODIGO.md
- **Resultado de Pruebas Unitarias Automatizadas:**

```
$ flutter test
```

```
00:00 +0: loading test/avistamiento_vm_test.dart
```

```
00:00 +0: Validación de avistamiento - Falla si la descripción está vacía
```

```
00:00 +1: Validación de avistamiento - Falla si no hay foto
```

```
00:00 +2: Validación de avistamiento - Falla si no tiene ubicación válida
```

```
00:00 +3: All tests passed!
```